

```
1 // ===== FILE: src/main.ts =====
2 /**
3  * 村口大战外星人 · 游戏入口
4  *
5  * 先出标准 Loading（插画 + 进度条 + 健康游戏忠告），资源就绪再落村子。
6  */
7 import '@core/pixiUnsafeEvalPatch';
8 import { analytics, initAnalytics, setAnalyticsUserId } from '@analytics';
9 import { SAVE_KEY } from '@config/CloudConfig';
10 import { BASE_GAME_KEY } from '@config/gameKeyScope';
11 import { BackendService } from '@core/BackendService';
12 import { CloudSyncManager } from '@core/CloudSyncManager';
13 import { EventBus } from '@core/EventBus';
14 import { Game } from '@core/Game';
15 import { PersistService } from '@core/PersistService';
16 import { SceneManager } from '@core/SceneManager';
17 import { Platform } from '@core/PlatformService';
18 import {
19   LOADING_SPLASH,
20   LOADING_TITLE,
21   preloadPaths,
22   villageArtPaths,
23 } from '@core/TextureLoader';
24 import { OverlayManager } from '@core/OverlayManager';
25 import { BgmPlayer } from '@core/BgmPlayer';
26 import { GMManager } from '@core/GMManager';
27 import { BattleScene } from '@scenes/BattleScene';
28 import { VillageScene } from '@scenes/VillageScene';
29 import { LoadingScreenOverlay } from '@ui/LoadingScreenOverlay';
30 import { GMEntryButton } from '@ui/GMEntryButton';
31 import { GMPanel } from '@ui/GMPanel';
32 declare const GameGlobal: any;
33 const MIN_LOADING_MS = 900;
34 function sleep(ms: number): Promise<void> {
35   return new Promise((resolve) => setTimeout(resolve, ms));
36 }
37 initAnalytics();
38 CloudSyncManager.prewarm();
39 PersistService.subscribeCloudImport((info) => {
40   if (!info.changedKeys.includes(SAVE_KEY)) return;
41   EventBus.emit('home:refresh');
42 });
43 if (typeof GameGlobal !== 'undefined') {
44   const prevError = GameGlobal.onError;
45   const prevReject = GameGlobal.onUnhandledRejection;
46   GameGlobal.onError = (msg: string) => {
47     console.error('[GlobalError]', msg);
48     try { prevError?.(msg); } catch { /**/ }
49     analytics.trackAppError(msg, { source: 'GameGlobal.onError' });
50   };
51 }
```

```
51 GameGlobal.onUnhandledRejection = (ev: { reason?: unknown }) => {
52   console.error('[UnhandledRejection]', ev?.reason || ev);
53   try { prevReject?.(ev); } catch { /* */ }
54   analytics.trackAppError(ev?.reason || ev, { source: 'unhandledRejection' });
55 };
56 }
57 async function resolveUserId(): Promise<string> {
58   const startupSync = await CloudSyncManager.awaitStartupSync();
59   console.log(`[main] 云同步启动结果: ${startupSync.status}, reason=${startupSync.reason}`);
60   let userId = CloudSyncManager.userId;
61   if (!userId && BackendService.available) {
62     try {
63       await BackendService.ensureToken();
64       userId = BackendService.userId;
65     } catch (error) {
66       console.warn('[main] 启动后补登失败 (不挡玩)', error);
67     }
68   }
69   return userId;
70 }
71 async function main(): Promise<void> {
72   const canvas = GameGlobal?.canvas ?? null;
73   if (!canvas) {
74     console.error('[main] 找不到 canvas');
75     return;
76   }
77   Game.init(canvas);
78   if (!Game.app?.renderer) {
79     console.error('[main] 渲染器初始化失败');
80     return;
81   }
82   console.log(`[main] 启动 平台=${Platform.name} gameKey=${BASE_GAME_KEY} 版本=${__APP_VERSION__}`);
83   const userIdP = resolveUserId();
84   Game.stage.sortableChildren = true;
85   const loading = new LoadingScreenOverlay();
86   Game.stage.addChild(loading);
87   Game.syncFrameToScreen();
88   const shownAt = Date.now();
89   await preloadPaths([LOADING_SPLASH, LOADING_TITLE]);
90   loading.applySplashTexture();
91   loading.applyTitleTexture();
92   loading.setProgress(0.12);
93   Game.syncFrameToScreen();
94   await preloadPaths(villageArtPaths(), (loaded, total) => {
95     const ratio = total > 0 ? loaded / total : 1;
96     loading.setProgress(0.12 + ratio * 0.72);
97   });
98   loading.setProgress(0.86);
99   const userId = await userIdP;
100  loading.setProgress(0.92);
```

```
101 SceneManager.register(new VillageScene());
102 SceneManager.register(new BattleScene());
103 SceneManager.switchTo('village');
104 if (GMMManager.isRuntimeAllowed) {
105     OverlayManager.container.addChild(new GMPanel());
106     OverlayManager.container.addChild(new GMEntryButton());
107 }
108 await Game.warmScenePresent();
109 loading.setProgress(1);
110 Game.syncFrameToScreen();
111 const wait = MIN_LOADING_MS - (Date.now() - shownAt);
112 if (wait > 0) await sleep(wait);
113 Game.stage.removeChild(loading);
114 loading.destroy({ children: true });
115 Game.syncFrameToScreen();
116 // game.js 的启动超时诊断靠这个标记判断是否真的画出来了
117 if (typeof GameGlobal !== 'undefined') GameGlobal.__gameRendered = true;
118 if (userId) {
119     setAnalyticsUserId(userId);
120 } else {
121     console.warn('[main] 未拿到登录 userId, 经分仅以 anonymous_id 上报);
122 }
123 analytics.trackSessionStart({
124     entry: 'main_boot',
125     with_user_id: !!userId,
126     cloud_sync_ready: CloudSyncManager.ready,
127 });
128 let lastHideAt = 0;
129 Platform.onHide() => {
130     BgmPlayer.pause();
131     void CloudSyncManager.flushNow('app-hide');
132     analytics.trackSessionEnd('app-hide');
133     lastHideAt = Date.now();
134 };
135 Platform.onShow() => {
136     BgmPlayer.resume();
137     if (lastHideAt > 0) {
138         analytics.trackAppShow({
139             background_ms: Date.now() - lastHideAt,
140         });
141     }
142 };
143 }
144 main().catch((e) => {
145     console.error('[main] 启动失败:', e);
146 });
147 // ===== FILE: src/core/PlatformService.ts =====
148 /**
149  * 平台服务抽象层 - 统一封装微信/抖音双平台 API
150  *
```

```
151 * 业务侧跨平台 SDK 入口：所有 wx/tt 差异（存储、登录、分享、生命周期等）
152 * 都必须走 Platform，禁止在业务里写 typeof wx / typeof tt。
153 *
154 * 宿主识别：抖音注入 tt（可同时存在 wx 兼容层）；微信仅 wx。
155 * 检测到哪个宿主，就只绑定该宿主原生 API（抖音→tt，微信→wx），互不倒用。
156 */
157 declare const wx: any;
158 declare const tt: any;
159 export type PlatformName = 'wechat' | 'douyin' | 'unknown';
160 export type BackendPlatformCode = 'wx' | 'dy' | 'anon';
161 /** 检测当前小游戏宿主（单一真源，与 minigame/runtime.js 逻辑一致） */
162 export function detectMinigamePlatform(): PlatformName {
163   if (typeof tt !== 'undefined') return 'douyin';
164   if (typeof wx !== 'undefined') return 'wechat';
165   return 'unknown';
166 }
167 /** 指定宿主的原生 API：抖音仅 tt，微信仅 wx */
168 export function getNativePlatformApi(platform: PlatformName = detectMinigamePlatform()): any {
169   if (platform === 'douyin') return typeof tt !== 'undefined' ? tt : null;
170   if (platform === 'wechat') return typeof wx !== 'undefined' ? wx : null;
171   return null;
172 }
173 /** @deprecated 请用 detectMinigamePlatform + getNativePlatformApi */
174 export function resolveMinigameRuntime(): { name: PlatformName; api: any } {
175   const name = detectMinigamePlatform();
176   return { name, api: getNativePlatformApi(name) };
177 }
178 export function toBackendPlatformCode(name: PlatformName): BackendPlatformCode {
179   if (name === 'douyin') return 'dy';
180   if (name === 'wechat') return 'wx';
181   return 'anon';
182 }
183 class PlatformServiceClass {
184   /** 当前平台名 */
185   readonly name: PlatformName;
186   /** 底层平台 API 对象（wx / tt / null） */
187   private _api: any;
188   /** adUnitId → 激励视频实例（宿主本身也是单例，这里避免重复注册回调） */
189   private _adCache = new Map<string, any>();
190   /** 当前在播广告的 Promise resolver，null = 无广告在播 */
191   private _adResolve: ((ok: boolean) => void) | null = null;
192   constructor() {
193     this.name = detectMinigamePlatform();
194     this._api = getNativePlatformApi(this.name);
195     console.log(`[Platform] 当前平台: ${this.name}, api=${this.name === 'douyin' ? 'tt' : this.name === ...
196   }
197   /** 是否在小游戏环境中 */
198   get isMinigame(): boolean {
199     return this._api !== null;
200   }
```

```
201 get isWechat(): boolean {
202     return this.name === 'wechat';
203 }
204 get isDouyin(): boolean {
205     return this.name === 'douyin';
206 }
207 /** 后端 login 接口 platform 字段 (wx / dy / anon) */
208 get backendPlatformCode(): BackendPlatformCode {
209     return toBackendPlatformCode(this.name);
210 }
211 /** 是否具备 HTTP 能力 (小游戏 request 或浏览器 fetch) */
212 get canUseBackend(): boolean {
213     return typeof this._api?.request === 'function' || typeof fetch === 'function';
214 }
215 /** 开发者工具 (非真机) */
216 get isDevtools(): boolean {
217     if (!this.isMinigame) return false;
218     try {
219         return this._api?.getSystemInfoSync()?.platform === 'devtools';
220     } catch {
221         return false;
222     }
223 }
224 /** 底层 API (慎用, 优先使用封装方法) */
225 get api(): any {
226     return this._api;
227 }
228 // 存储
229 getStorageSync(key: string): string | null {
230     try {
231         return this._api?.getStorageSync(key) || null;
232     } catch (_) {
233         return null;
234     }
235 }
236 setStorageSync(key: string, value: string): void {
237     try {
238         this._api?.setStorageSync(key, value);
239     } catch (_) {}
240 }
241 /** 异步写入本地存储 (避免阻塞主线程) */
242 setStorageAsync(key: string, value: string): void {
243     try {
244         if (this._api?.setStorage) {
245             this._api.setStorage({ key, data: value, fail() {} });
246         } else {
247             this._api?.setStorageSync(key, value);
248         }
249     } catch (_) {}
250 }
```

```
251 removeStorageSync(key: string): void {
252   try {
253     this._api?.removeStorageSync(key);
254   } catch (_) {}
255 }
256 getSystemInfoSync(): Record<string, unknown> {
257   try {
258     return this._api?.getSystemInfoSync?(). ?? {};
259   } catch {
260     return {};
261   }
262 }
263 /** 经分 SDK / 后端 HTTP 请求 (Promise 风格) */
264 request(opts: {
265   url: string;
266   method?: 'GET' | 'POST' | 'PUT' | 'DELETE';
267   data?: unknown;
268   headers?: Record<string, string>;
269   timeoutMs?: number;
270 }): Promise<{ statusCode: number; data: unknown }> {
271   const method = (opts.method || 'POST').toUpperCase();
272   const headers: Record<string, string> = {
273     'content-type': 'application/json',
274     ...(opts.headers || {}),
275   };
276   const timeoutMs = opts.timeoutMs && opts.timeoutMs > 0 ? opts.timeoutMs : 10000;
277   const payload = opts.data === undefined || typeof opts.data === 'string'
278     ? opts.data
279     : JSON.stringify(opts.data);
280   if (this._api?.request) {
281     return new Promise((resolve, reject) => {
282       let done = false;
283       const timer = setTimeout(() => {
284         if (done) return;
285         done = true;
286         reject(new Error(`request timeout: ${opts.url}`));
287       }, timeoutMs);
288       try {
289         this._api.request({
290           url: opts.url,
291           method,
292           data: payload,
293           header: headers,
294           timeout: timeoutMs,
295           success: (res: { statusCode?: number; data?: unknown }) => {
296             if (done) return;
297             done = true;
298             clearTimeout(timer);
299             resolve({ statusCode: res?.statusCode ?? 0, data: res?.data });
300           },
```

```
301     fail: (err: unknown) => {
302         if (done) return;
303         done = true;
304         clearTimeout(timer);
305         reject(err);
306     },
307 });
308 } catch (e) {
309     if (!done) {
310         done = true;
311         clearTimeout(timer);
312         reject(e);
313     }
314 }
315 });
316 }
317 if (typeof fetch === 'function') {
318     const controller = new AbortController();
319     const timer = setTimeout(() => controller.abort(), timeoutMs);
320     return fetch(opts.url, {
321         method,
322         headers,
323         body: payload as BodyInit | undefined,
324         signal: controller.signal,
325     })
326     .then(async (res) => {
327         clearTimeout(timer);
328         const text = await res.text();
329         let data: unknown = text;
330         try { data = text ? JSON.parse(text) : null; } catch { /* keep text */ }
331         return { statusCode: res.status, data };
332     })
333     .catch((e) => {
334         clearTimeout(timer);
335         throw e;
336     });
337 }
338 return Promise.reject(new Error('no http transport available'));
339 }
340 /** 平台登录 code (wx.login / tt.login) */
341 loginCode(): Promise<string> {
342     return new Promise((resolve) => {
343         if (!this._api?.login) {
344             resolve("");
345             return;
346         }
347         try {
348             this._api.login({
349                 success: (res: { code?: string }) => resolve(res?.code || ""),
350                 fail: () => resolve(""),
```

```
351     });
352   } catch {
353     resolve("");
354   }
355   });
356 }
357 // 创建资源
358 /** 创建平台 Image 对象（加载本地/网络图片用） */
359 createImage(): any {
360   if (this._api?.createImage) return this._api.createImage();
361   if (typeof Image !== 'undefined') return new Image();
362   return null;
363 }
364 /** 下载远程文件到临时路径（CDN 资源用） */
365 downloadFile(url: string): Promise<{ tempFilePath?: string; statusCode?: number }> {
366   return new Promise((resolve, reject) => {
367     if (!this._api?.downloadFile) {
368       reject(new Error('downloadFile unavailable'));
369       return;
370     }
371     this._api.downloadFile({
372       url,
373       success: (res: { tempFilePath?: string; statusCode?: number }) => {
374         const statusCode = Number(res?.statusCode || 0);
375         if (statusCode > 0 && (statusCode < 200 || statusCode >= 300)) {
376           reject(new Error(`downloadFile status=${statusCode} url=${url}`));
377           return;
378         }
379         if (!res?.tempFilePath) {
380           reject(new Error(`downloadFile missing tempFilePath url=${url}`));
381           return;
382         }
383         resolve(res);
384       },
385       fail: (err: any) => {
386         reject(new Error(err?.errMsg || err?.message || String(err)));
387       },
388     });
389   });
390 }
391 /** 创建 InnerAudioContext（BGM / 音效） */
392 createInnerAudioContext(): WechatMinigame.InnerAudioContext | null {
393   try {
394     return this._api?.createInnerAudioContext?.() ?? null;
395   } catch {
396     return null;
397   }
398 }
399 // 交互反馈
400 /** 短振动（消除/点击反馈），type 控制强度 */
```



```

401  vibrateShort(type: 'light' | 'medium' | 'heavy' = 'light'): void {
402    try {
403      this._api?.vibrateShort?({ type });
404    } catch (_) {}
405  }
406  /** 长振动（重击/Boss 登场等强反馈） */
407  vibrateLong(): void {
408    try {
409      this._api?.vibrateLong?();
410    } catch (_) {}
411  }
412  showToast(title: string, icon: 'success' | 'error' | 'none' = 'none'): void {
413    try {
414      this._api?.showToast?({ title, icon });
415    } catch (_) {}
416  }
417  /** 自定义分析。有宿主走 reportEvent，没有就打日志，切片验收够用。 */
418  reportEvent(name: string, data: Record<string, unknown> = {}): void {
419    try {
420      if (typeof this._api?.reportEvent === 'function') {
421        this._api.reportEvent(name, data);
422        return;
423      }
424    } catch {
425      /* 上报失败不挡玩 */
426    }
427    try {
428      console.log(`[track] ${name}`, data);
429    } catch {
430      /* */
431    }
432  }
433  /** 关掉当前原生 toast（激励广告关闭后宿主偶发自带提示时用） */
434  hideToast(): void {
435    try {
436      this._api?.hideToast?();
437    } catch (_) {}
438  }
439  /**
440   * 激励视频广告。已配置 createRewardedVideoAd 则拉起；
441   * 否则开发/本地环境短暂提示后视为成功，便于联调各广告位。
442   *
443   * 广告实例按 adUnitId 缓存并只注册一次回调：宿主的 createRewardedVideoAd 对同一
444   * adUnitId 返回同一单例，每次播放都重新 onClose 会让回调越积越多，
445   * 同一次关闭被回调 N 次（第二次以后打到已结束的 Promise 上，静默丢失奖励）。
446   */
447  showRewardedVideo(adUnitId = ""): Promise<boolean> {
448    return new Promise((resolve) => {
449      // 同一时刻只允许一支广告在播，避免两个入口的奖励串到一起
450      if (this._adResolve) {

```

```
451     resolve(false);
452     return;
453 }
454 // 开发者工具里也能 new 出广告实例，但关广告经常不带 isEnded，
455 // 联调会误判没看完。工具里走桩，真机才拉真实激励视频。
456 if (this.isDevtools) {
457     this.showToast('广告播放中...');
458     setTimeout(() => {
459         this.hideToast();
460         resolve(true);
461     }, 700);
462     return;
463 }
464 let ad: any = null;
465 try {
466     ad = this._rewardedAd(adUnitId);
467 } catch (_) { /* fall through mock */ }
468 if (!ad) {
469     this.showToast('广告播放中...');
470     setTimeout(() => {
471         // 桩结束立刻清掉，避免和业务侧翻倍数额动画叠在一起
472         this.hideToast();
473         resolve(true);
474     }, 700);
475     return;
476 }
477 this._adResolve = resolve;
478 try {
479     const p = ad.show();
480     // 拉取失败先 load 再播一次：小游戏侧常见于弱网首次拉取超时
481     if (p?.catch) {
482         p.catch(() => {
483             const l = ad.load?();
484             if (l?.then) l.then(() => ad.show()).catch(() => this._settleAd(false));
485             else this._settleAd(false);
486         });
487     }
488 } catch (_) {
489     this._settleAd(false);
490 }
491 });
492 }
493 private _rewardedAd(adUnitId: string): any {
494     const cached = this._adCache.get(adUnitId);
495     if (cached) return cached;
496     const create = this._api?.createRewardedVideoAd;
497     if (typeof create !== 'function') return null;
498     const ad = create.call(this._api, { adUnitId });
499     if (!ad?.onClose || !ad?.show) return null;
500     ad.onClose((res: { isEnded?: boolean }) => this._settleAd(!res?.isEnded));
```

```
501   ad.onError?.(() => this._settleAd(false));
502   this._adCache.set(adUnitId, ad);
503   return ad;
504 }
505 private _settleAd(ok: boolean): void {
506   const resolve = this._adResolve;
507   this._adResolve = null;
508   resolve?.(ok);
509 }
510 /**
511  * 插屏广告（抖音流量主 / 广告金政策建议接入）。
512  * 未配置 adUnitId、宿主不支持、加载失败 → resolve(false)，不挡流程。
513  * 抖音要求「展示成功后再播须 destroy 再建」：每次调用新建实例，关闭后销毁。
514  */
515 showInterstitialAd(adUnitId = ""): Promise<boolean> {
516   return new Promise((resolve) => {
517     if (!adUnitId) {
518       resolve(false);
519       return;
520     }
521     const create = this._api?.createInterstitialAd;
522     if (typeof create !== 'function') {
523       resolve(false);
524       return;
525     }
526     let ad: any = null;
527     let settled = false;
528     let shown = false;
529     const finish = (ok: boolean): void => {
530       if (settled) return;
531       settled = true;
532       try { ad?.destroy?.(); } catch { /* ignore */ }
533       resolve(ok);
534     };
535     try {
536       ad = create.call(this._api, { adUnitId });
537     } catch {
538       resolve(false);
539       return;
540     }
541     if (!ad?.show) {
542       resolve(false);
543       return;
544     }
545     ad.onError?.(() => finish(false));
546     ad.onClose?.(() => finish(true));
547     ad.onLoad?.(() => {
548       try {
549         const p = ad.show();
550         if (p?.then) {
```

```

551     p.then() => { shown = true; }).catch() => finish(false));
552   } else {
553     shown = true;
554   }
555   } catch {
556     finish(false);
557   }
558 });
559 // 创建后会自动 load; 超时仍未展示则放行 (避免卡死跳转)
560 setTimeout() => {
561   if (!settled && !shown) finish(false);
562 }, 5000);
563 });
564 }
565 /**
566  * 订阅消息 (抖音广告金政策建议接入) 。
567  * 必须在用户点击/支付回调里调用; tmplIds 来自后台「运营-订阅消息」。
568  * 返回各模板 id → accept|reject|ban|fail; 失败或未配置返回空对象。
569  */
570 requestSubscribeMessage(tmplIds: readonly string[]): Promise<Record<string, string>> {
571   return new Promise((resolve) => {
572     const ids = tmplIds.filter(Boolean).slice(0, 3);
573     const api = this._api?.requestSubscribeMessage;
574     if (typeof api !== 'function' || ids.length === 0) {
575       resolve({});
576       return;
577     }
578     try {
579       api.call(this._api, {
580         tmplIds: ids,
581         success: (res: Record<string, string>) => resolve(res ?? {}),
582         fail: () => resolve({}),
583         complete: () => { /* success/fail 已 settle */ },
584       });
585     } catch {
586       resolve({});
587     }
588   });
589 }
590 showModal(title: string, content: string): void {
591   try {
592     this._api?.showModal?({ title, content, showCancel: false });
593   } catch (_) {}
594 }
595 // 分享
596 showShareMenu(opts?: { withShareTicket?: boolean; menus?: string[] }): void {
597   try {
598     this._api?.showShareMenu?({
599       withShareTicket: opts?.withShareTicket ?? true,
600       menus: opts?.menus ?? ['shareAppMessage', 'shareTimeline'],

```

```
601     });
602   } catch (_) {}
603 }
604 shareAppMessage(opts: { title: string; imageUrl?: string; query?: string }): void {
605   try {
606     this._api?.shareAppMessage?.(opts);
607   } catch (_) {}
608 }
609 onShareAppMessage(callback: () => { title: string; imageUrl?: string; query?: string }): void {
610   try {
611     this._api?.onShareAppMessage?.(callback);
612   } catch (_) {}
613 }
614 onShareTimeline(callback: () => { title: string; imageUrl?: string; query?: string }): void {
615   try {
616     this._api?.onShareTimeline?.(callback);
617   } catch (_) {}
618 }
619 // 生命周期
620 onShow(handler: (opts: any) => void): void {
621   try {
622     this._api?.onShow?.(handler);
623   } catch (_) {}
624 }
625 onHide(handler: () => void): void {
626   try {
627     this._api?.onHide?.(handler);
628   } catch (_) {}
629 }
630 // 抖音侧边栏复访
631 /** 检测宿主是否支持指定场景（如 sidebar） */
632 checkScene(opts: {
633   scene: string;
634   success?: (res: { isExist?: boolean }) => void;
635   fail?: (err?: unknown) => void;
636 }): void {
637   try {
638     if (this._api?.checkScene) {
639       this._api.checkScene(opts);
640     } else {
641       opts.fail?.({ errMsg: 'checkScene not supported' });
642     }
643   } catch (e) {
644     opts.fail?.(e);
645   }
646 }
647 /** 跳转宿主场景（侧边栏复访必接） */
648 navigateToScene(opts: {
649   scene: string;
650   success?: () => void;
```

```
651   fail?: (err?: unknown) => void;
652 }): void {
653   try {
654     if (this._api?.navigateToScene) {
655       this._api.navigateToScene(opts);
656     } else {
657       opts.fail?({ errMsg: 'navigateToScene not supported' });
658     }
659   } catch (e) {
660     opts.fail?.(e);
661   }
662 }
663 // 抖音添加到桌面 (广告金政策必接)
664 /** 检查桌面快捷方式是否已添加 (仅 Android 有效) */
665 checkShortcut(opts: {
666   success?: (res: { status?: { exist?: boolean; needUpdate?: boolean } }) => void;
667   fail?: (err?: unknown) => void;
668 }): void {
669   try {
670     if (this._api?.checkShortcut) {
671       this._api.checkShortcut(opts);
672     } else {
673       opts.fail?({ errMsg: 'checkShortcut not supported' });
674     }
675   } catch (e) {
676     opts.fail?.(e);
677   }
678 }
679 /**
680  * 添加小游戏到手机桌面 (须在用户点击/touchend 内同步调用)
681  * 仅支持抖音 / 抖音极速版 / 抖音火山版
682  */
683 addShortcut(opts: {
684   success?: () => void;
685   fail?: (err?: { errMsg?: string }) => void;
686   complete?: () => void;
687 }): void {
688   try {
689     if (this._api?.addShortcut) {
690       this._api.addShortcut(opts);
691     } else {
692       opts.fail?({ errMsg: 'addShortcut not supported' });
693     }
694   } catch (e) {
695     opts.fail?.(e as { errMsg?: string });
696   }
697 }
698 /**
699  * 加载本地自定义字体。
700  * 优先 wx/tt.loadFont(path) (同步返回 family) ; 否则走 loadFontFace。
```

```
701  * 非小游戏环境返回 null, 由 FontService 用 @font-face 兜底。
702  */
703  loadFont(path: string, family: string): Promise<string | null> {
704    if (!this.isMinigame) return Promise.resolve(null);
705    // 微信/抖音小游戏: loadFont 最稳, 直接返回可用 family 名
706    if (typeof this._api?.loadFont === 'function') {
707      try {
708        const loaded = this._api.loadFont(path) as string | undefined;
709        return Promise.resolve(loaded || family);
710      } catch (e) {
711        console.warn('[Platform] loadFont 失败', path, e);
712      }
713    }
714    if (typeof this._api?.loadFontFace === 'function') {
715      return new Promise((resolve) => {
716        try {
717          this._api.loadFontFace({
718            family,
719            source: `url("${path}")`,
720            global: true,
721            success: () => resolve(family),
722            fail: (err: unknown) => {
723              console.warn('[Platform] loadFontFace 失败', path, err);
724              resolve(null);
725            },
726          });
727        } catch (e) {
728          console.warn('[Platform] loadFontFace 异常', path, e);
729          resolve(null);
730        }
731      });
732    }
733    return Promise.resolve(null);
734  }
735 }
736 export const Platform = new PlatformServiceClass();
737 // ===== FILE: src/core/BackendService.ts =====
738 import {
739   BACKEND_ANON_ID_KEY,
740   BACKEND_BASE_URL,
741   BACKEND_LOGIN_PATH,
742   BACKEND_PULL_PATH,
743   BACKEND_PUSH_PATH,
744   BACKEND_REQUEST_TIMEOUT_MS,
745   BACKEND_TOKEN_KEY,
746 } from '@config/CloudConfig';
747 import { Platform } from '@core/PlatformService';
748 export interface BackendPullResult {
749   userId: string;
750   platform: string;
```

```
751   exists: boolean;
752   schemaVersion: number;
753   updatedAt: number;
754   payload: Record<string, string>;
755   payloadKeys: string[];
756   clientFingerprint?: string;
757 }
758 export interface BackendPushPayload {
759   schemaVersion: number;
760   updatedAt: number;
761   baseRemoteUpdatedAt: number;
762   clientFingerprint: string;
763   payload: Record<string, string>;
764 }
765 export interface BackendPushResult {
766   userId: string;
767   updatedAt: number;
768   savedAt: number;
769   mode: 'insert' | 'update';
770   sizeBytes: number;
771 }
772 interface StoredToken {
773   token: string;
774   userId: string;
775   platform: string;
776   expiresAt: number;
777 }
778 export class BackendError extends Error {
779   readonly status: number;
780   readonly code: string;
781   readonly data?: unknown;
782   constructor(status: number, code: string, message: string, data?: unknown) {
783     super(message);
784     this.name = 'BackendError';
785     this.status = status;
786     this.code = code;
787     this.data = data;
788   }
789 }
790 class BackendServiceClass {
791   private stored: StoredToken | null = null;
792   private loginInflight: Promise<StoredToken> | null = null;
793   get available(): boolean {
794     return Platform.canUseBackend;
795   }
796   get userId(): string {
797     return this.stored?.userId || '';
798   }
799   async ensureToken(): Promise<StoredToken> {
800     if (this.stored && this.stored.expiresAt - Date.now() > 60_000) {
```



```
801     if (this.isTokenForCurrentPlatform(this.stored)) {
802         return this.stored;
803     }
804     console.warn('[Backend] 缓存 token 平台不匹配(${this.stored.platform}→${this.expectedPlatform()}), ...
805     this.clearToken();
806 }
807 const cached = this.loadTokenFromStorage();
808 if (cached && cached.expiresAt - Date.now() > 60_000) {
809     if (this.isTokenForCurrentPlatform(cached)) {
810         this.stored = cached;
811         return cached;
812     }
813     console.warn('[Backend] 本地 token 平台不匹配(${cached.platform}→${this.expectedPlatform()}), 重新登录');
814     this.clearToken();
815 }
816 if (this.loginInflight) {
817     return this.loginInflight;
818 }
819 this.loginInflight = this.login().finally(() => {
820     this.loginInflight = null;
821 });
822 return this.loginInflight;
823 }
824 pullSave(): Promise<BackendPullResult> {
825     return this.callWithAuth<BackendPullResult>(BACKEND_PULL_PATH, {});
826 }
827 pushSave(snapshot: BackendPushPayload): Promise<BackendPushResult> {
828     return this.callWithAuth<BackendPushResult>(BACKEND_PUSH_PATH, snapshot);
829 }
830 clearToken(): void {
831     this.stored = null;
832     Platform.removeStorageSync(BACKEND_TOKEN_KEY);
833 }
834 private async login(): Promise<StoredToken> {
835     const body = await this.buildLoginBody();
836     console.log('[Backend] login start platform=${body.platform}');
837     const { status, data } = await this.request(BACKEND_LOGIN_PATH, body, undefined);
838     const res = data as { ok?: boolean; code?: string; error?: string; data?: Partial<StoredToken...
839     if (status !== 200 || !res || res.ok !== true || !res.data?.token) {
840         const code = res?.code || 'LOGIN_FAIL';
841         const msg = res?.error || `login failed (status=${status})`;
842         console.warn('[Backend] login fail platform=${body.platform} code=${code} msg=${msg}');
843         throw new BackendError(status, code, msg, res?.data);
844     }
845     const stored = {
846         token: String(res.data.token),
847         userId: String(res.data.userId || ''),
848         platform: String(res.data.platform || body.platform),
849         expiresAt: Number(res.data.expiresAt || 0),
850     };
```

```
851   this.stored = stored;
852   Platform.setStorageSync(BACKEND_TOKEN_KEY, JSON.stringify(stored));
853   console.log(`[Backend] login ok platform=${stored.platform} userId=${stored.userId}`);
854   return stored;
855 }
856 private expectedPlatform(): string {
857   return Platform.backendPlatformCode;
858 }
859 private isTokenForCurrentPlatform(stored: StoredToken): boolean {
860   const expected = this.expectedPlatform();
861   const actual = stored.platform || stored.userId.split(':')[0] || '';
862   return actual === expected;
863 }
864 private async buildLoginBody(): Promise<{ platform: string; code?: string; anonId?: string }> {
865   const platform = Platform.backendPlatformCode;
866   if (platform === 'wx' || platform === 'dy') {
867     const code = await Platform.loginCode();
868     if (!code) {
869       const errCode = platform === 'wx' ? 'NO_WX_CODE' : 'NO_TT_CODE';
870       throw new BackendError(0, errCode, `${platform} login did not return code`);
871     }
872     return { platform, code };
873   }
874   return { platform: 'anon', anonId: this.getOrCreateAnonId() };
875 }
876 private getOrCreateAnonId(): string {
877   const existing = Platform.getStorageSync(BACKEND_ANON_ID_KEY);
878   if (existing) {
879     return existing;
880   }
881   const id = `anon_${Date.now().toString(36)}_${Math.random().toString(36).slice(2, 10)}`;
882   Platform.setStorageSync(BACKEND_ANON_ID_KEY, id);
883   return id;
884 }
885 private async callWithAuth<T>(path: string, body: unknown): Promise<T> {
886   const token = await this.ensureToken();
887   const { status, data } = await this.request(path, body, token.token);
888   if (status === 401) {
889     this.clearToken();
890     const retryToken = await this.ensureToken();
891     const retry = await this.request(path, body, retryToken.token);
892     return this.unwrap<T>(retry.status, retry.data);
893   }
894   return this.unwrap<T>(status, data);
895 }
896 private unwrap<T>(status: number, data: unknown): T {
897   const res = data as { ok?: boolean; code?: string; error?: string; data?: T };
898   if (status === 200 && res?.ok === true) {
899     return res.data as T;
900   }
```

```
901   const code = res?.code || `HTTP_${status}`;
902   const msg = res?.error || `request failed status=${status}`;
903   throw new BackendError(status, code, msg, res?.data);
904 }
905 private async request(
906   path: string,
907   body: unknown,
908   token: string | undefined,
909 ): Promise<{ status: number; data: unknown }> {
910   const headers: Record<string, string> = {
911     'content-type': 'application/json',
912   };
913   if (token) {
914     headers.authorization = `Bearer ${token}`;
915   }
916   const res = await Platform.request({
917     url: BACKEND_BASE_URL + path,
918     method: 'POST',
919     data: body || {},
920     headers,
921     timeoutMs: BACKEND_REQUEST_TIMEOUT_MS,
922   });
923   return { status: res.statusCode, data: res.data };
924 }
925 private loadTokenFromStorage(): StoredToken | null {
926   const raw = Platform.getStorageSync(BACKEND_TOKEN_KEY);
927   if (!raw) {
928     return null;
929   }
930   try {
931     const parsed = JSON.parse(raw) as Partial<StoredToken>;
932     if (!parsed.token) {
933       return null;
934     }
935     return {
936       token: parsed.token,
937       userId: parsed.userId || '',
938       platform: parsed.platform || '',
939       expiresAt: Number(parsed.expiresAt || 0),
940     };
941   } catch {
942     return null;
943   }
944 }
945 }
946 export const BackendService = new BackendServiceClass();
947 // ===== FILE: src/core/CloudSyncManager.ts =====
948 import {
949   CLOUD_SYNC_BASE_DELAY_MS,
950   CLOUD_SYNC_DEBOUNCE_MS,
```

```
951 CLOUD_SYNC_LOG_THRESHOLD,
952 CLOUD_SYNC_MAX_BACKOFF_MS,
953 CLOUD_SYNC_MAX_FAIL_COUNT,
954 CLOUD_SYNC_RETRY_INTERVAL_MS,
955 CLOUD_SYNC_STARTUP_TIMEOUT_MS,
956 } from '@config/CloudConfig';
957 import { BackendError, BackendService } from '@core/BackendService';
958 import { PersistService } from '@core/PersistService';
959 import { Platform } from '@core/PlatformService';
960 export type CloudAuthorityState = 'disabled' | 'unknown' | 'confirmedRemote' | 'cacheOnly';
961 export interface CloudStartupSyncResult {
962   status: 'disabled' | 'confirmed' | 'remote-applied' | 'cache-only';
963   reason: string;
964 }
965 class CloudSyncManagerClass {
966   private initPromise: Promise<void> | null = null;
967   private startupPromise: Promise<void> | null = null;
968   private cloudReady = false;
969   private initDone = false;
970   private syncTimer: ReturnType<typeof setTimeout> | null = null;
971   private retryTimer: ReturnType<typeof setInterval> | null = null;
972   private syncFailCount = 0;
973   private syncDisabled = false;
974   private syncing = false;
975   private syncPending = false;
976   private authorityState: CloudAuthorityState = this.enabled ? 'unknown' : 'disabled';
977   private lastStartupRemoteApplied = false;
978   constructor() {
979     PersistService.subscribe((changedKeys) => {
980       if (changedKeys.length > 0) {
981         this.scheduleSync(`dirty:${changedKeys.length}`);
982       }
983     });
984   }
985   get enabled(): boolean {
986     return BackendService.available;
987   }
988   get ready(): boolean {
989     return this.cloudReady;
990   }
991   get userId(): string {
992     return BackendService.userId;
993   }
994   prewarm(): void {
995     if (!this.enabled) {
996       return;
997     }
998     if (!this.startupPromise) {
999       this.startupPromise = this.initialize();
1000     }
  }
```

```
1001 }
1002 async awaitStartupSync(timeoutMs = CLOUD_SYNC_STARTUP_TIMEOUT_MS): Promise<CloudStartupSyncResu...
1003 if (!this.enabled) {
1004     this.authorityState = 'disabled';
1005     return { status: 'disabled', reason: 'backend-disabled' };
1006 }
1007 this.prewarm();
1008 if (!this.startupPromise) {
1009     return { status: 'disabled', reason: 'startup-missing' };
1010 }
1011 let timer: ReturnType<typeof setTimeout> | null = null;
1012 const result = await Promise.race([
1013     this.startupPromise.then(() => 'done' as const).catch(() => 'done' as const),
1014     new Promise<void>((resolve) => {
1015         timer = setTimeout(resolve, timeoutMs);
1016     }).then(() => 'timeout' as const),
1017 ]);
1018 if (timer) {
1019     clearTimeout(timer);
1020 }
1021 if (result === 'timeout') {
1022     return { status: 'cache-only', reason: 'startup-timeout' };
1023 }
1024 if (this.authorityState === 'cacheOnly') {
1025     return { status: 'cache-only', reason: 'startup-pull-failed' };
1026 }
1027 return {
1028     status: this.lastStartupRemoteApplied ? 'remote-applied' : 'confirmed',
1029     reason: this.lastStartupRemoteApplied ? 'remote-imported' : 'cloud-confirmed',
1030 };
1031 }
1032 scheduleSync(reason = 'debounce'): void {
1033     if (!this.enabled) {
1034         return;
1035     }
1036     this.prewarm();
1037     if (this.authorityState === 'cacheOnly') {
1038         this.syncPending = true;
1039         return;
1040     }
1041     if (!this.initDone) {
1042         this.syncPending = true;
1043         return;
1044     }
1045     if (!this.cloudReady || this.syncDisabled) {
1046         return;
1047     }
1048     if (this.syncTimer) {
1049         clearTimeout(this.syncTimer);
1050     }
```

```
1051   const delay = this.syncFailCount > 0
1052   ? Math.min(CLOUD_SYNC_BASE_DELAY_MS * Math.pow(2, this.syncFailCount - 1), CLOUD_SYNC_MAX_B...
1053   : CLOUD_SYNC_DEBOUNCE_MS;
1054   this.syncTimer = setTimeout(() => {
1055     this.syncTimer = null;
1056     void this.syncToCloud(reason);
1057   }, delay);
1058 }
1059 async flushNow(reason = 'manual'): Promise<void> {
1060   if (!this.enabled) {
1061     return;
1062   }
1063   this.prewarm();
1064   if (this.startupPromise) {
1065     try {
1066       await this.startupPromise;
1067     } catch {
1068       // 初始化失败已经走本地缓存
1069     }
1070   }
1071   if (this.authorityState === 'cacheOnly' || !this.cloudReady) {
1072     return;
1073   }
1074   if (this.syncTimer) {
1075     clearTimeout(this.syncTimer);
1076     this.syncTimer = null;
1077   }
1078   await this.syncToCloud(reason, true);
1079 }
1080 private async initialize(): Promise<void> {
1081   if (this.initPromise) {
1082     return this.initPromise;
1083   }
1084   this.initPromise = (async () => {
1085     try {
1086       await BackendService.ensureToken();
1087       this.cloudReady = !!BackendService.userId;
1088       if (!this.cloudReady) {
1089         this.enterCacheOnly('no-user-id');
1090         return;
1091       }
1092       await this.pullFromCloudOnStartup();
1093     } catch (error) {
1094       console.warn('[CloudSync] init failed, using local cache', error);
1095       this.enterCacheOnly('init-failed');
1096     } finally {
1097       this.initDone = true;
1098       if (this.syncPending && this.cloudReady) {
1099         this.syncPending = false;
1100         this.scheduleSync('pending-after-init');
```

```
1101     }
1102   }
1103   });
1104   return this.initPromise;
1105 }
1106 private async pullFromCloudOnStartup(): Promise<void> {
1107   let remote;
1108   try {
1109     remote = await BackendService.pullSave();
1110   } catch (error) {
1111     console.warn('[CloudSync] startup pull failed, keeping local cache', error);
1112     this.enterCacheOnly('startup-pull-failed');
1113     return;
1114   }
1115   const hasLocalData = PersistService.hasAnyLocalCloudData();
1116   if (!remote.exists || Object.keys(remote.payload || {}).length === 0) {
1117     // 远端空档：保留本地进度，标脏后推上去。不能按空 payload 覆盖本地。
1118     this.confirmRemoteBaseline(0, remote.exists ? 'startup-empty-remote-doc' : 'startup-no-remo...
1119     this.lastStartupRemoteApplied = false;
1120     if (hasLocalData) {
1121       PersistService.touchCloudMeta();
1122       this.scheduleSync('startup-seed-remote');
1123     }
1124     return;
1125   }
1126   const localMeta = PersistService.getCloudSyncMeta();
1127   const remoteUpdatedAt = Number(remote.updatedAt) || 0;
1128   const hasKnownRemoteBaseline = localMeta.remoteUpdatedAt > 0;
1129   const shouldApplyRemote = !hasLocalData || !hasKnownRemoteBaseline || remoteUpdatedAt > local...
1130   if (shouldApplyRemote) {
1131     PersistService.importCloudSnapshot({
1132       updatedAt: remoteUpdatedAt,
1133       payload: remote.payload || {},
1134       reason: 'startup',
1135     });
1136     this.confirmRemoteBaseline(remoteUpdatedAt, 'startup-remote-imported');
1137     this.lastStartupRemoteApplied = true;
1138     return;
1139   }
1140   this.confirmRemoteBaseline(remoteUpdatedAt, 'startup-remote-confirmed');
1141   if (PersistService.isCloudDirty()) {
1142     this.scheduleSync('startup-local-dirty');
1143   }
1144 }
1145 private async syncToCloud(reason: string, force = false): Promise<void> {
1146   if (!this.cloudReady || (!force && this.syncDisabled) || this.authorityState === 'cacheOnly') {
1147     return;
1148   }
1149   if (this.syncing) {
1150     this.syncPending = true;
```

```
1151     return;
1152   }
1153   this.syncing = true;
1154   try {
1155     const snapshot = PersistService.exportCloudSnapshot();
1156     if (!PersistService.isCloudDirty() || snapshot.payloadKeys.length === 0) {
1157       return;
1158     }
1159     if (snapshot.updatedAt <= 0) {
1160       PersistService.touchCloudMeta();
1161     }
1162     const finalSnapshot = PersistService.exportCloudSnapshot();
1163     try {
1164       const res = await BackendService.pushSave({
1165         schemaVersion: finalSnapshot.schemaVersion,
1166         updatedAt: finalSnapshot.updatedAt,
1167         baseRemoteUpdatedAt: finalSnapshot.baseRemoteUpdatedAt,
1168         clientFingerprint: this.buildClientFingerprint(),
1169         payload: finalSnapshot.payload,
1170       });
1171       PersistService.markCloudSynced(res.updatedAt || finalSnapshot.updatedAt);
1172       this.confirmRemoteBaseline(res.updatedAt || finalSnapshot.updatedAt, `push-ok:${reason}`);
1173       this.syncFailCount = 0;
1174       this.syncDisabled = false;
1175       if (this.retryTimer) {
1176         clearInterval(this.retryTimer);
1177         this.retryTimer = null;
1178       }
1179     } catch (error) {
1180       if (error instanceof BackendError && error.code === 'STALE_UPDATE' && error.data && typeo...
1181         const remote = (error.data as { remote?: { updatedAt?: number; payload?: Record<string,...
1182         PersistService.importCloudSnapshot({
1183           updatedAt: Number(remote?.updatedAt) || Date.now(),
1184           payload: remote?.payload || {},
1185           reason: 'stale-update',
1186         });
1187         this.confirmRemoteBaseline(Number(remote?.updatedAt) || Date.now(), 'stale-update');
1188         this.syncFailCount = 0;
1189         this.syncDisabled = false;
1190         return;
1191       }
1192       throw error;
1193     }
1194   } catch (error: unknown) {
1195     this.syncFailCount += 1;
1196     const message = error instanceof Error ? error.message : String(error);
1197     if (this.syncFailCount <= CLOUD_SYNC_LOG_THRESHOLD) {
1198       console.warn(`[CloudSync] push failed (${this.syncFailCount}/${CLOUD_SYNC_MAX_FAIL_COUNT})...
1199     }
1200     if (this.syncFailCount >= CLOUD_SYNC_MAX_FAIL_COUNT) {
```



```

1201     this.syncDisabled = true;
1202     if (!this.retryTimer) {
1203         this.retryTimer = setInterval(() => {
1204             if (!this.syncing && PersistService.isCloudDirty()) {
1205                 void this.syncToCloud('retry-interval', true);
1206             }
1207         }, CLOUD_SYNC_RETRY_INTERVAL_MS);
1208     }
1209 } else if (PersistService.isCloudDirty()) {
1210     this.scheduleSync('retry-after-fail:${reason}');
1211 }
1212 } finally {
1213     this.syncing = false;
1214     if (this.syncPending && !this.syncDisabled) {
1215         this.syncPending = false;
1216         this.scheduleSync('pending-resume');
1217     }
1218 }
1219 }
1220 private buildClientFingerprint(): string {
1221     const info = Platform.getSystemInfoSync();
1222     return [
1223         Platform.name,
1224         info.brand,
1225         info.model,
1226         info.platform,
1227     ].map((v) => (v == null ? '' : String(v))).filter(Boolean).join('|').slice(0, 160);
1228 }
1229 private enterCacheOnly(reason: string): void {
1230     if (this.authorityState === 'cacheOnly') {
1231         return;
1232     }
1233     this.authorityState = 'cacheOnly';
1234     console.warn(`[CloudSync] cacheOnly reason=${reason}`);
1235 }
1236 private confirmRemoteBaseline(remoteUpdatedAt: number, reason: string): void {
1237     this.authorityState = 'confirmedRemote';
1238     console.log(`[CloudSync] remote baseline confirmed reason=${reason}, remoteUpdatedAt=${remote...`);
1239 }
1240 }
1241 export const CloudSyncManager = new CloudSyncManagerClass();
1242 // ===== FILE: src/core/RunMemory.ts =====
1243 /**
1244  * 存档: 村民名单、进化、星级、村庄等级、四种资源、弹子、推图进度、布阵。
1245  *
1246  * 一条硬口径 (§ 5) : **货币只有四种** —— 废铁 / 零件 / 工分 / 村庄经验。
1247  * 弹子是次数不是货币, 所以它跟资源分开存, 也不进货币条。
1248  * 想加第五种之前先回去看反目标第五条。
1249  *
1250  * 这一版把上一版的字段全换了 (改装件、门路研发、难度阶梯、废品堆全部下线),

```

```
1251 * 所以老存档不迁移、直接重置: `REV` 不匹配就当新档。
1252 * 迁移的成本远高于收益 —— 上一版的养成对象 (27 件破烂的星级)
1253 * 在这一版压根不存在, 硬折算只会折出一个玩家看不懂的开局。
1254 */
1255 import { SAVE_KEY } from '@config/CloudConfig';
1256 import { PersistService } from '@core/PersistService';
1257 import {
1258   PELLET_AD, PELLET_AD_DAILY, PELLET_CLEAR, PELLET_FIRST, PELLET_LOSE,
1259   SETTLE_SCRAP, SETTLE_SCRAP_REPLAY, creditPity, mulberry32, pelletCap,
1260   pelletRegenMin, shoot,
1261   type ShotResult,
1262 } from '@balance/stall';
1263 import {
1264   CALL_COST, addVillageExp, clampVillageLv, craftOf, evoFromCraft,
1265   nextFeed, rollCall, squadCap, starsOf, yieldMul,
1266   type Progress,
1267 } from '@balance/village';
1268 import {
1269   DEFAULT_SQUAD, STAR_MAX, VILLAGERS, VILLAGER_BY_ID,
1270 } from '@balance/villagers';
1271 import { STAGE_COUNT, clampStage, getStage } from '@balance/stages';
1272 import { CELL_COUNT, LANE_COUNT } from '@balance/combat';
1273 const KEY = SAVE_KEY;
1274 /** 存档格式版本。改字段就往上加一, 老档直接重置 */
1275 const REV = 3;
1276 /** 战场上一个人的位置。存下来是为了下一关不用重排 */
1277 export interface Slot {
1278   id: string;
1279   lane: number;
1280   cell: number;
1281 }
1282 export interface RunMemory {
1283   rev: number;
1284   /* ---- 养成 ---- */
1285   villageLv: number;
1286   villageExp: number;
1287   /** 已入伙的村民 id, 按入伙顺序 */
1288   roster: string[];
1289   /** 每人几阶 (1~3) 。由手艺推导, 存着给旧档和面板用 */
1290   evo: Record<string, number>;
1291   /** 每人手艺 1~10。缺字段时按 evo 反推 */
1292   craft: Record<string, number>;
1293   /** 每人几颗星 */
1294   stars: Record<string, number>;
1295   scrap: number;
1296   parts: number;
1297   credits: number;
1298   /* ---- 弹弓摊 ---- */
1299   /** 手上还有几发 */
1300   pellets: number;
```

```
1301  /** 上次结算离线回弹的时刻 */
1302  pelletAtMs: number;
1303  /** 摊子的工分保底计数（离上次出工分过了几发） */
1304  stallPity: number;
1305  /** 今天摊子广告看了几次，以及那是哪一天 */
1306  adCount: number;
1307  adDay: string;
1308  /* ---- 推图 ---- */
1309  stageId: number;
1310  stageTop: number;
1311  /** 每关拿到过的最高星评 */
1312  stageStars: Record<number, number>;
1313  /* ---- 布阵 ---- */
1314  /** 上一次的排法。开下一关时直接铺上 */
1315  layout: Slot[];
1316  /* ---- 其他 ---- */
1317  /** 一共喊过几次人。前 4 次必出新人的保底靠它 */
1318  callCount: number;
1319  /** 见过的人，点亮图鉴用（含已经不在名单里的，虽然目前不会掉人） */
1320  seenIds: string[];
1321  }
1322  function empty(): RunMemory {
1323    return {
1324      rev: REV,
1325      villageLv: 1,
1326      villageExp: 0,
1327      roster: [...DEFAULT_SQUAD],
1328      evo: {},
1329      craft: {},
1330      stars: {},
1331      scrap: 0,
1332      parts: 0,
1333      credits: 0,
1334      pellets: 6,
1335      pelletAtMs: Date.now(),
1336      stallPity: 0,
1337      adCount: 0,
1338      adDay: "",
1339      stageId: 1,
1340      stageTop: 1,
1341      stageStars: {},
1342      layout: [],
1343      callCount: 0,
1344      seenIds: [...DEFAULT_SQUAD],
1345    };
1346  }
1347  function validIds(raw: unknown, fallback: readonly string[]): string[] {
1348    if (!Array.isArray(raw)) return [...fallback];
1349    const out = raw.filter(
1350      (id): id is string => typeof id === 'string' && VILLAGER_BY_ID[id] !== undefined,
```

```

1351 );
1352 return out.length > 0 ? [...new Set(out)] : [...fallback];
1353 }
1354 function numMap(raw: unknown, lo: number, hi: number): Record<string, number> {
1355   const out: Record<string, number> = {};
1356   if (!raw || typeof raw !== 'object') return out;
1357   for (const [k, v] of Object.entries(raw as Record<string, unknown>)) {
1358     if (!VILLAGER_BY_ID[k]) continue;
1359     const n = Math.floor(Number(v) || 0);
1360     if (n > lo) out[k] = Math.min(hi, n);
1361   }
1362   return out;
1363 }
1364 function validLayout(raw: unknown, roster: readonly string[]): Slot[] {
1365   if (!Array.isArray(raw)) return [];
1366   const owned = new Set(roster);
1367   const used = new Set<string>();
1368   const out: Slot[] = [];
1369   for (const r of raw) {
1370     const s = r as Partial<Slot>;
1371     if (typeof s.id !== 'string' || !owned.has(s.id)) continue;
1372     const lane = Math.floor(Number(s.lane) || 0);
1373     const cell = Math.floor(Number(s.cell) || 0);
1374     if (lane < 0 || lane >= LANE_COUNT || cell < 0 || cell >= CELL_COUNT) continue;
1375     const key = `${lane},${cell}`;
1376     if (used.has(key) || out.some((o) => o.id === s.id)) continue;
1377     used.add(key);
1378     out.push({ id: s.id, lane, cell });
1379   }
1380   return out;
1381 }
1382 export function loadMemory(): RunMemory {
1383   try {
1384     const raw = PersistService.readRaw(KEY);
1385     if (!raw) return empty();
1386     const p = JSON.parse(raw) as Partial<RunMemory>;
1387     // 上一版的存档字段这一版全没有，硬折算只会折出看不懂的开局
1388     if (Number(p.rev) !== REV) return empty();
1389     const roster = validIds(p.roster, DEFAULT_SQUAD);
1390     const stageTop = clampStage(p.stageTop);
1391     const mem: RunMemory = {
1392       rev: REV,
1393       villageLv: clampVillageLv(p.villageLv),
1394       villageExp: Math.max(0, Number(p.villageExp) || 0),
1395       roster,
1396       evo: numMap(p.evo, 1, 3),
1397       craft: numMap(p.craft, 1, 10),
1398       stars: numMap(p.stars, 0, STAR_MAX),
1399       scrap: Math.max(0, Number(p.scrap) || 0),
1400       parts: Math.max(0, Number(p.parts) || 0),

```

```

1401   credits: Math.max(0, Number(p.credits) || 0),
1402   pellets: Math.max(0, Number(p.pellets) || 0),
1403   pelletAtMs: Math.max(0, Number(p.pelletAtMs) || Date.now()),
1404   stallPity: Math.max(0, Number(p.stallPity) || 0),
1405   adCount: Math.max(0, Number(p.adCount) || 0),
1406   adDay: typeof p.adDay === 'string' ? p.adDay : '',
1407   stageTop,
1408   stageId: Math.min(clampStage(p.stageId), stageTop),
1409   stageStars: (() => {
1410     const out: Record<number, number> = {};
1411     const src = p.stageStars;
1412     if (src && typeof src === 'object') {
1413       for (const [k, v] of Object.entries(src as Record<string, unknown>)) {
1414         const id = Number(k);
1415         if (id >= 1 && id <= STAGE_COUNT) {
1416           out[id] = Math.max(0, Math.min(3, Math.floor(Number(v) || 0)));
1417         }
1418       }
1419     }
1420     return out;
1421   })(),
1422   layout: validLayout(p.layout, roster),
1423   callCount: Math.max(0, Number(p.callCount) || 0),
1424   seenIds: validIds(p.seenIds, roster),
1425 };
1426 return mem;
1427 } catch {
1428   return empty();
1429 }
1430 }
1431 function persist(next: RunMemory): RunMemory {
1432   const out = { ...next, rev: REV };
1433   try {
1434     PersistService.writeRaw(KEY, JSON.stringify(out));
1435   } catch {
1436     /* 模拟器偶发写失败，不挡继续玩 */
1437   }
1438   return out;
1439 }
1440 /** 存档摊成引擎和面板都能读的 Progress */
1441 export function progressOf(mem: RunMemory): Progress {
1442   return {
1443     villageLv: mem.villageLv,
1444     villageExp: mem.villageExp,
1445     roster: mem.roster,
1446     evo: mem.evo,
1447     craft: mem.craft,
1448     stars: mem.stars,
1449     scrap: mem.scrap,
1450     parts: mem.parts,

```

```
1451   credits: mem.credits,
1452   };
1453 }
1454 /** 这一局能上几个人 */
1455 export function capOf(mem: RunMemory): number {
1456   return squadCap(mem.villageLv);
1457 }
1458 /* ----- 弹子 ----- */
1459 /**
1460  * 把离线攒的弹子结算到现在。
1461  *
1462  * 只回到 pelletCap, 不许越过 —— 弹子是次数, 攒着不玩没有额外好处,
1463  * 这是「每天回来打一會兒」和「攒一周一次性刷完」之间的那道闸。
1464  */
1465 export function settlePellets(nowMs: number = Date.now()): RunMemory {
1466   const prev = loadMemory();
1467   const cap = pelletCap(prev.villageLv);
1468   if (prev.pelletAtMs <= 0) return persist({ ...prev, pelletAtMs: nowMs });
1469   const perMs = pelletRegenMin(prev.villageLv) * 60_000;
1470   const gained = Math.floor((nowMs - prev.pelletAtMs) / perMs);
1471   if (gained <= 0) return prev;
1472   const pellets = Math.min(cap, prev.pellets + gained);
1473   return persist({
1474     ...prev,
1475     pellets,
1476     // 余数留着, 别让频繁进出把零头抹掉
1477     pelletAtMs: prev.pelletAtMs + gained * perMs,
1478   });
1479 }
1480 /** 打一发。没弹子返回 undefined */
1481 export function shootStall(nowMs: number = Date.now()): {
1482   mem: RunMemory;
1483   result: ShotResult;
1484 } | undefined {
1485   const prev = settlePellets(nowMs);
1486   if (prev.pellets <= 0) return undefined;
1487   const rng = mulberry32((nowMs ^ (prev.pellets * 2654435761)) >>> 0);
1488   const { result, pityCount } = shoot(rng, prev.villageLv, prev.stallPity);
1489   const grown = addVillageExp(progressOf(prev), Math.round(result.gain.exp));
1490   const mem = persist({
1491     ...prev,
1492     pellets: prev.pellets - 1,
1493     stallPity: pityCount,
1494     scrap: prev.scrap + Math.round(result.gain.scrap),
1495     parts: prev.parts + Math.round(result.gain.parts),
1496     credits: prev.credits + Math.round(result.gain.credits),
1497     villageLv: grown.lv,
1498     villageExp: grown.exp,
1499   });
1500   return { mem, result };
```

```
15923 names: readonly UiName[],
15924 cx: number,
15925 cy: number,
15926 h: number,
15927 gap = 6,
15928 ): boolean {
15929   const texts = names.map((n) => uiTex(n));
15930   if (texts.some((t) => !t?.baseTexture.valid || t.width <= 1)) return false;
15931   const ws = texts.map((t) => (t!.width / t!.height) * h);
15932   const total = ws.reduce((a, b) => a + b, 0) + gap * Math.max(0, texts.length - 1);
15933   let x = cx - total / 2;
15934   texts.forEach((t, i) => {
15935     fitSprite(parent, t, x + ws[i]! / 2, cy, ws[i]!, h);
15936     x += ws[i]! + gap;
15937   });
15938   return true;
15939 }
15940 export function numGlyphs(n: number): UiName[] {
15941   return String(Math.floor(Math.max(0, n))).split("").map((d) => `paint_${d}` as UiName);
15942 }
15943 /** 3/5 这种比分。数字走漆字，斜杠没有贴图就手写一笔。 */
15944 export function paintFrac(
15945   parent: PIXI.Container,
15946   a: number,
15947   b: number,
15948   cx: number,
15949   cy: number,
15950   h: number,
15951 ): boolean {
15952   const left = numGlyphs(a);
15953   const right = numGlyphs(b);
15954   const names = [...left, ...right];
15955   const texts = names.map((n) => uiTex(n));
15956   if (texts.some((t) => !t?.baseTexture.valid || t.width <= 1)) return false;
15957   const slashW = h * 0.28;
15958   const gap = 3;
15959   const ws = texts.map((t) => (t!.width / t!.height) * h);
15960   const leftW = ws.slice(0, left.length).reduce((s, w) => s + w, 0) + gap * Math.max(0, left.leng...
15961   const rightW = ws.slice(left.length).reduce((s, w) => s + w, 0) + gap * Math.max(0, right.lengt...
15962   const total = leftW + slashW + rightW + gap * 2;
15963   let x = cx - total / 2;
15964   texts.slice(0, left.length).forEach((t, i) => {
15965     fitSprite(parent, t, x + ws[i]! / 2, cy, ws[i]!, h);
15966     x += ws[i]! + gap;
15967   });
15968   const slash = painted(Math.round(h * 0.72), GOLD, '#1a1008', 3);
15969   slash.anchor.set(0.5);
15970   slash.position.set(x + slashW / 2, cy);
15971   slash.text = '/';
15972   parent.addChild(slash);
```

```

15973 x += slashW + gap;
15974 teks.slice(left.length).forEach((t, i) => {
15975     const w = ws[left.length + i];
15976     fitSprite(parent, t, x + w / 2, cy, w, h);
15977     x += w + gap;
15978 });
15979 return true;
15980 }
15981 // ===== FILE: src/ui/lintel.ts =====
15982 /**
15983  * 村子门楣只给主界面用：大牌 + 经验槽 + 四枚资源章。
15984  * 编队 / 战斗另用切下来的上半块锈铁（battleHudLay），不要把经验条和四格硬塞进去。
15985  */
15986 export interface LintelLay {
15987     titleH: number;
15988     title: { cx: number; cy: number; w: number; h: number };
15989     titleGlyphH: number;
15990     /** 中间凹槽中线，经验条 / 门路字落在这儿 */
15991     exp: { x: number; y: number; w: number };
15992     hintY: number;
15993     stamp: { y: number; w: number; h: number; cx: readonly number[] };
15994     barBottom: number;
15995 }
15996 export function lintelLay(safeTop: number, height: number): LintelLay {
15997     const safe = Math.max(safeTop, 16);
15998     let titleH = Math.round(750 * 9 / 16);
15999     if (titleH > height * 0.32) titleH = Math.round(height * 0.32);
16000     if (titleH < safe + 200) titleH = Math.min(Math.round(height * 0.34), safe + 240);
16001     const slotTop = titleH * 0.35;
16002     const slotMid = titleH * 0.405;
16003     const plateTop = titleH * 0.10;
16004     let glyphH = Math.max(44, Math.round(titleH * 0.15));
16005     let titleCy = (plateTop + slotTop) / 2 + 12;
16006     if (titleCy + glyphH / 2 > slotTop - 10) {
16007         titleCy = slotTop - 10 - glyphH / 2;
16008     }
16009     if (titleCy - glyphH / 2 < 8) {
16010         glyphH = Math.max(32, (titleCy - 8) * 2);
16011     }
16012     return {
16013         titleH,
16014         title: { cx: 375, cy: titleCy, w: 750, h: titleH },
16015         titleGlyphH: glyphH,
16016         exp: { x: 78, y: slotMid, w: 594 },
16017         hintY: slotTop + titleH * 0.12,
16018         stamp: {
16019             y: titleH * 0.71,
16020             w: 750 * 0.20,
16021             h: titleH * 0.20,
16022             cx: [750 * 0.141, 750 * 0.379, 750 * 0.615, 750 * 0.854],

```



```
16023     },
16024     barBottom: titleH,
16025   };
16026 }
16027 /**
16028  * 编队 / 战斗顶板。底只铺切下来的上半块锈铁，
16029  * 上场 / 来 / 波 等字和锈章是另出的图，叠在板上，不烤进底板。
16030  *
16031  * 切图是 1280×398，750 宽时约 233 高；为了塞下三枚章会略往下拉。
16032  */
16033 export interface BattleHudLay {
16034   titleH: number;
16035   title: { cx: number; cy: number };
16036   titleGlyphH: number;
16037   hintY: number;
16038   stamp: { y: number; w: number; h: number; cxs: readonly number[] };
16039   barBottom: number;
16040 }
16041 export function battleHudLay(safeTop: number, height: number): BattleHudLay {
16042   const safe = Math.max(safeTop, 16);
16043   let titleH = Math.round(750 * 398 / 1280);
16044   if (titleH < safe + 200) titleH = Math.min(Math.round(height * 0.24), safe + 220);
16045   if (titleH > height * 0.24) titleH = Math.round(height * 0.24);
16046   const titleGlyphH = Math.max(32, Math.round(titleH * 0.16));
16047   let titleCy = titleH * 0.24;
16048   if (titleCy - titleGlyphH / 2 < safe + 4) {
16049     titleCy = safe + 4 + titleGlyphH / 2;
16050   }
16051   const hintY = titleH - 16;
16052   const stampH = Math.round(titleH * 0.30);
16053   const stampY = hintY - 12 - stampH / 2;
16054   return {
16055     titleH,
16056     title: { cx: 375, cy: titleCy },
16057     titleGlyphH,
16058     hintY,
16059     stamp: {
16060       y: stampY,
16061       w: 200,
16062       h: stampH,
16063       cxs: [155, 375, 595],
16064     },
16065     barBottom: titleH,
16066   };
16067 }
16068 // ===== FILE: src/ui/paint.ts =====
16069 /**
16070  * 切片统一画法。
16071  * 角色站在地面上，UI 用暗底金边；禁止方框套圆框、色块套立绘。
16072  */
```

```
16073 import * as PIXI from 'pixi.js';
16074 export const GOLD = 0xc9a46a;
16075 export const INK = 0x0c0e14;
16076 export const PLATE = 0x14161f;
16077 /** 村民一人一色，暖色系。外星人用冷色，两边一眼分得开 */
16078 const VILLAGER_COLOR: Readonly<Record<string, number>> = {
16079   tiezhu: 0xc4703a,
16080   dachui: 0xd9a13b,
16081   laoli: 0xb4553f,
16082   erjiu: 0xa8823f,
16083   sanshen: 0xd4736b,
16084   laoyanqiang: 0x8f7a4a,
16085 };
16086 export function label(size: number, color = 0xffffffff, bold = false): PIXI.Text {
16087   return new PIXI.Text("", {
16088     fontFamily: 'sans-serif',
16089     fontSize: size,
16090     fontWeight: bold ? 'bold' : 'normal',
16091     fill: color,
16092   });
16093 }
16094 /** 锈金漆字。门楣上的关卡名、资源数走这一套，不要裸系统字。 */
16095 export function painted(size: number, color: number, rim = '#1a1008', thick = 4): PIXI.Text {
16096   const t = label(size, color, true);
16097   t.style.stroke = rim;
16098   t.style.strokeThickness = thick;
16099   return t;
16100 }
16101 export function plate(
16102   g: PIXI.Graphics,
16103   x: number,
16104   y: number,
16105   w: number,
16106   h: number,
16107   radius = 14,
16108   alpha = 0.78,
16109 ): void {
16110   g.beginFill(PLATE, alpha).drawRoundedRect(x, y, w, h, radius).endFill();
16111   g.lineStyle(1.5, GOLD, 0.4).drawRoundedRect(x, y, w, h, radius).lineStyle(0);
16112 }
16113 export function goldBtn(g: PIXI.Graphics, x: number, y: number, w: number, h: number): void {
16114   g.beginFill(0x2a2218, 0.94).drawRoundedRect(x, y, w, h, 16).endFill();
16115   g.lineStyle(2, GOLD, 0.85).drawRoundedRect(x, y, w, h, 16).lineStyle(0);
16116 }
16117 export function rivet(g: PIXI.Graphics, x: number, y: number, r = 5): void {
16118   oldNail(g, x, y, r);
16119 }
16120 function hash01(n: number): number {
16121   const x = Math.sin(n * 127.1 + 311.7) * 43758.5453;
16122   return x - Math.floor(x);
```

```
16123 }
16124 /** 铜锈钉头: 褐铜底、橙锈边、偶尔一点铜绿 */
16125 export function oldNail(g: PIXI.Graphics, x: number, y: number, r = 8, seed = 1): void {
16126     const rust = hash01(seed + 3);
16127     g.beginFill(0x0a0604, 0.42).drawCircle(x + 1.3, y + 2.3, r).endFill();
16128     g.beginFill(0x3a2214).drawCircle(x, y, r).endFill();
16129     g.beginFill(rust > 0.62 ? 0x4e8c7a : 0xc45a22, rust > 0.62 ? 0.42 : 0.55)
16130         .drawEllipse(x + r * 0.12, y + r * 0.18, r * 0.82, r * 0.7)
16131         .endFill();
16132     g.beginFill(0x5a3a22).drawCircle(x - r * 0.1, y - r * 0.16, r * 0.58).endFill();
16133     g.beginFill(0xe8c080, 0.42).drawCircle(x - r * 0.3, y - r * 0.34, r * 0.2).endFill();
16134 }
16135 /**
16136  * 顶边钉子。两头簇着钉，中间疏密不一，高低错开，不要一条尺上的点。
16137  */
16138 export function nailRow(g: PIXI.Graphics, x: number, y: number, w: number, r = 8): void {
16139     const spots = [
16140         0.045, 0.078, 0.108,
16141         0.21, 0.34, 0.49, 0.61, 0.76,
16142         0.89, 0.922, 0.955,
16143     ];
16144     spots.forEach((t, i) => {
16145         const nx = x + w * t + (hash01(i + 4) - 0.5) * 16;
16146         const ny = y + (hash01(i + 17) - 0.5) * 14 + (i < 3 || i > 7 ? (hash01(i + 29) - 0.3) * 8 : 0);
16147         const rr = r * (0.72 + hash01(i + 41) * 0.55);
16148         oldNail(g, nx, ny, rr, i + 7);
16149     });
16150 }
16151 /** 角上三颗簇钉，跟着圆角落，不要排成等边三角 */
16152 export function nailCluster(g: PIXI.Graphics, cx: number, cy: number, r = 6, seed = 0): void {
16153     const offs = [
16154         [-16, 4],
16155         [2, -8],
16156         [14, 6],
16157     ];
16158     offs.forEach(([dx, dy], i) => {
16159         oldNail(
16160             g,
16161             cx + dx + (hash01(seed + i) - 0.5) * 8,
16162             cy + dy + (hash01(seed + i + 5) - 0.5) * 7,
16163             r * (0.75 + hash01(seed + i + 9) * 0.45),
16164             seed + i,
16165         );
16166     });
16167 }
16168 /** 局外贴图：整件缩进框里，脚和链子都留着，不裁。 */
16169 export function fitSprite(
16170     parent: PIXI.Container,
16171     texture: PIXI.Texture | null,
16172     cx: number,
```

```

16173   cy: number,
16174   maxW: number,
16175   maxH: number,
16176 ): PIXI.Sprite | null {
16177   if (!texture?.baseTexture.valid || texture.width <= 1) return null;
16178   const spr = new PIXI.Sprite(texture);
16179   spr.anchor.set(0.5);
16180   spr.position.set(cx, cy);
16181   spr.scale.set(Math.min(maxW / texture.width, maxH / texture.height));
16182   spr.eventMode = 'none';
16183   parent.addChild(spr);
16184   return spr;
16185 }
16186 /**
16187  * 脚钉在同一条线上。三阶并排时传入同一 scale,
16188  * 免得武器把画布拉高之后人被越缩越小。
16189  */
16190 export function standSprite(
16191   parent: PIXI.Container,
16192   texture: PIXI.Texture | null,
16193   cx: number,
16194   feetY: number,
16195   maxW: number,
16196   maxH: number,
16197   scale?: number,
16198 ): PIXI.Sprite | null {
16199   if (!texture?.baseTexture.valid || texture.width <= 1) return null;
16200   const spr = new PIXI.Sprite(texture);
16201   spr.anchor.set(0.5, 1);
16202   spr.scale.set(scale ?? Math.min(maxW / texture.width, maxH / texture.height));
16203   spr.position.set(cx, feetY);
16204   spr.eventMode = 'none';
16205   parent.addChild(spr);
16206   return spr;
16207 }
16208 /** 木板铺满面板，多出来的边裁掉。 */
16209 /** 铜锈斑。贴图被压平了也还能看见橙锈和铜绿 */
16210 export function copperRust(
16211   g: PIXI.Graphics,
16212   x: number,
16213   y: number,
16214   w: number,
16215   h: number,
16216 ): void {
16217   const blobs: readonly { rx: number; ry: number; rw: number; rh: number; c: number; a: number }[...]
16218     { rx: 0.06, ry: 0.08, rw: 0.22, rh: 0.1, c: 0xc45a22, a: 0.28 },
16219     { rx: 0.68, ry: 0.04, rw: 0.24, rh: 0.11, c: 0xb85a2a, a: 0.26 },
16220     { rx: 0.38, ry: 0.42, rw: 0.2, rh: 0.09, c: 0xd46a28, a: 0.16 },
16221     { rx: 0.12, ry: 0.62, rw: 0.18, rh: 0.14, c: 0x4e8c7a, a: 0.16 },
16222     { rx: 0.72, ry: 0.7, rw: 0.2, rh: 0.12, c: 0xc46a28, a: 0.2 },

```

```
16223   { rx: 0.48, ry: 0.12, rw: 0.12, rh: 0.07, c: 0x5aa08b, a: 0.12 },
16224   { rx: 0.22, ry: 0.28, rw: 0.14, rh: 0.08, c: 0x8a3a18, a: 0.18 },
16225 ];
16226 for (const b of blobs) {
16227   g.beginFill(b.c, b.a).drawEllipse(x + w * b.rx, y + h * b.ry, w * b.rw, h * b.rh).endFill();
16228 }
16229 }
16230 /** 锈铁板。主页顶栏和底坞用同一块料，不再混木牌和黑玻璃。 */
16231 export function ironSlab(
16232   g: PIXI.Graphics,
16233   x: number,
16234   y: number,
16235   w: number,
16236   h: number,
16237   radius = 12,
16238 ): void {
16239   g.beginFill(0x0a0604, 0.5).drawRoundedRect(x + 4, y + 6, w, h, radius).endFill();
16240   g.beginFill(0x2a1810).drawRoundedRect(x, y, w, h, radius).endFill();
16241   g.beginFill(0x3d2418, 0.72).drawRoundedRect(x + 7, y + 7, w - 14, h - 14, Math.max(4, radius - ...
16242   g.lineStyle(2, 0x6a4a28, 0.4).drawRoundedRect(x + 2, y + 2, w - 4, h - 4, radius).lineStyle(0);
16243   const inset = Math.min(18, w * 0.06, h * 0.22);
16244   rivet(g, x + inset, y + inset);
16245   rivet(g, x + w - inset, y + inset);
16246   rivet(g, x + inset, y + h - inset);
16247   rivet(g, x + w - inset, y + h - inset);
16248 }
16249 /**
16250  * 格子的地面垫。开打之后才画，布阵阶段画的是空心格（那时候格线本身就是热区）。
16251  * 只画在自家那 12 格上，上方空场仍然不铺——空场是舞台，见 BattleScene 的头注释。
16252  */
16253 export function queuePad(
16254   g: PIXI.Graphics,
16255   cx: number,
16256   feetY: number,
16257   opts: { empty: boolean; hot: boolean; front: boolean },
16258 ): void {
16259   const rx = opts.empty ? 18 : 16;
16260   const ry = opts.empty ? 7 : 6;
16261   g.beginFill(0x000000, opts.empty ? 0.22 : 0.18);
16262   g.drawEllipse(cx, feetY + 8, rx, ry);
16263   g.endFill();
16264   if (opts.hot) {
16265     g.lineStyle(3, 0x9be08a, 0.95).drawEllipse(cx, feetY + 8, rx + 4, ry + 3).lineStyle(0);
16266     g.beginFill(0x9be08a, 0.12).drawEllipse(cx, feetY + 8, rx, ry).endFill();
16267     return;
16268   }
16269   if (opts.empty) {
16270     g.lineStyle(2, GOLD, 0.55).drawEllipse(cx, feetY + 8, rx, ry).lineStyle(0);
16271     g.lineStyle(1.2, GOLD, 0.25).drawEllipse(cx, feetY + 8, rx * 0.62, ry * 0.62).lineStyle(0);
16272     return;
```

```
16273 }
16274 if (opts.front) {
16275   g.beginFill(GOLD, 0.18).drawEllipse(cx, feetY + 8, rx + 8, ry + 5).endFill();
16276   g.lineStyle(3.6, GOLD, 0.95).drawEllipse(cx, feetY + 8, rx + 7, ry + 4).lineStyle(0);
16277   g.lineStyle(1.4, 0xffe08a, 0.75).drawEllipse(cx, feetY + 8, rx + 1, ry + 1).lineStyle(0);
16278 }
16279 }
16280 /**
16281  * 够得到哪儿：贴地椭圆，不是竖条。近战是围着人的一小片，远程铺到最远点。
16282  *
16283  * 布阵阶段每个人都画一层淡的。§ 8.1 记着那个 bug —— 上一版战场太长而射程太短，
16284  * 后排两格打不到任何东西，而屏幕上完全看不出来。这一层就是为了让
16285  * 「他站这儿够不够得着」变成看一眼的事，而不是打完一局才发现。
16286  */
16287 export function rangeArea(
16288   g: PIXI.Graphics,
16289   cx: number,
16290   feetY: number,
16291   reachY: number,
16292   color: number,
16293   melee: boolean,
16294 ): void {
16295   const forward = Math.max(40, Math.abs(feetY - reachY));
16296   if (melee) {
16297     const rx = Math.max(78, forward * 1.2);
16298     const ry = Math.max(52, forward * 0.9);
16299     const cy = feetY - ry * 0.28;
16300     g.beginFill(color, 0.18).drawEllipse(cx, cy, rx, ry).endFill();
16301     g.lineStyle(2.6, color, 0.78).drawEllipse(cx, cy, rx, ry).lineStyle(0);
16302     g.lineStyle(1.2, 0xffffffff, 0.32).drawEllipse(cx, cy, rx * 0.9, ry * 0.9).lineStyle(0);
16303     return;
16304   }
16305   const ry = forward * 0.52;
16306   const rx = Math.min(300, Math.max(120, forward * 0.4));
16307   const cy = (feetY + reachY) / 2;
16308   g.beginFill(color, 0.1).drawEllipse(cx, cy, rx, ry).endFill();
16309   g.lineStyle(2.2, color, 0.55).drawEllipse(cx, cy, rx, ry).lineStyle(0);
16310 }
16311 /**
16312  * 经验条。玩家唯一能看见的「还差多久捡下一件破烂」。
16313  *
16314  * 攒满了不画成空条：破烂发完之后这条还在，得让它看起来是完成态而不是坏了。
16315  */
16316 export function expBar(
16317   g: PIXI.Graphics,
16318   x: number,
16319   y: number,
16320   width: number,
16321   ratio: number,
16322   done: boolean,
```

```
16323 ): void {
16324     const h = 12;
16325     const r = 6;
16326     g.beginFill(0x000000, 0.5).drawRoundedRect(x, y, width, h, r).endFill();
16327     const w = done ? width : Math.max(0, Math.min(1, ratio)) * width;
16328     if (w > 0) {
16329         g.beginFill(done ? 0x9be08a : GOLD, 0.95).drawRoundedRect(x, y, w, h, r).endFill();
16330     }
16331     g.lineStyle(1.2, GOLD, 0.45).drawRoundedRect(x, y, width, h, r).lineStyle(0);
16332 }
16333 /** 铺满一块矩形。顶栏锈牌、出村铁门要拉到设计尺寸，不能按原图比例缩成小方块。 */
16334 export function fillSprite(
16335     parent: PIXI.Container,
16336     texture: PIXI.Texture | null,
16337     cx: number,
16338     cy: number,
16339     w: number,
16340     h: number,
16341 ): PIXI.Sprite | null {
16342     if (!texture?.baseTexture.valid || texture.width <= 1) return null;
16343     const spr = new PIXI.Sprite(texture);
16344     spr.anchor.set(0.5);
16345     spr.position.set(cx, cy);
16346     spr.scale.set(w / texture.width, h / texture.height);
16347     spr.eventMode = 'none';
16348     parent.addChild(spr);
16349     return spr;
16350 }
16351 /** 钉在泥地里的木牌。关卡桩、能升阶木牌的兜底，贴图没来时也能认。 */
16352 export function woodPlank(
16353     g: PIXI.Graphics,
16354     x: number,
16355     y: number,
16356     w: number,
16357     h: number,
16358     radius = 8,
16359 ): void {
16360     g.beginFill(0x1a1008, 0.45).drawRoundedRect(x + 3, y + 5, w, h, radius).endFill();
16361     g.beginFill(0x5a3a18).drawRoundedRect(x, y, w, h, radius).endFill();
16362     g.beginFill(0x6e4a22, 0.78).drawRoundedRect(x + 5, y + 4, w - 10, h - 8, Math.max(3, radius - 3...
16363     g.lineStyle(2, 0x3a2410, 0.55).drawRoundedRect(x + 1, y + 1, w - 2, h - 2, radius).lineStyle(0);
16364 }
16365 export function hpBar(
16366     g: PIXI.Graphics,
16367     cx: number,
16368     y: number,
16369     width: number,
16370     ratio: number,
16371     color: number,
16372     barH = 6,
```

```

16373 ): void {
16374   const w = Math.max(0, Math.min(1, ratio)) * width;
16375   const r = Math.max(1, Math.round(barH / 2));
16376   g.beginFill(0x000000, 0.55).drawRoundedRect(cx - width / 2, y, width, barH, r).endFill();
16377   if (w > 0) g.beginFill(color, 0.95).drawRoundedRect(cx - width / 2, y, w, barH, r).endFill();
16378 }
16379 // ===== FILE: src/utils/bindPointerTap.ts =====
16380 /**
16381  * 可点击绑定（见 @/minigame/index.ts 统一交互层）：
16382  * - 微信小游戏：canvasTapRouter（touchstart/touchend + 设计坐标 hitTest）
16383  * - 纯浏览器：pointerdown + pointerup
16384  * 可选 onLongPress：按住约 450ms 触发，松手不再走短按。
16385  */
16386 import * as PIXI from 'pixi.js';
16387 import { Platform } from '@core/PlatformService';
16388 import { playSfx } from '@core/SfxPlayer';
16389 import { registerCanvasTap } from './canvasTapRouter';
16390 import { clientEventToDesign } from './clientEventToDesign';
16391 import { deferAfterPointerEvent } from './deferAfterPointer';
16392 const DEFAULT_LONG_PRESS_MS = 450;
16393 const HOLD_SLOP = 14;
16394 export function bindPointerTap(
16395   target: PIXI.Container,
16396   /** 参数是本次 tap 的设计坐标；只关心「点了」的调用方直接忽略即可。 */
16397   fn: (dx: number, dy: number) => void,
16398   opts?: {
16399     guard?: () => boolean;
16400     blockTap?: () => boolean;
16401     pointGuard?: (dx: number, dy: number) => boolean;
16402     sync?: boolean;
16403     /** 不播点击音（长按菜单、全屏挡板） */
16404     silent?: boolean;
16405     onLongPress?: () => void;
16406     longPressMs?: number;
16407   },
16408 ): void {
16409   let fired = false;
16410   const run = (dx: number, dy: number): void => {
16411     if (fired) return;
16412     if (opts?.guard && !opts.guard()) return;
16413     if (opts?.blockTap?.()) return;
16414     fired = true;
16415     if (opts?.sync) {
16416       try { fn(dx, dy); } catch (err) { console.error('[bindPointerTap sync]', err); }
16417       fired = false;
16418       return;
16419     }
16420     deferAfterPointerEvent(() => {
16421       fired = false;
16422       fn(dx, dy);

```



```
16423     });
16424   };
16425   if (Platform.isMinigame) {
16426     if (target.eventMode === 'none' || !target.eventMode) {
16427       target.eventMode = 'static';
16428     }
16429     registerCanvasTap({
16430       target,
16431       fn: run,
16432       guard: opts?.guard,
16433       blockTap: opts?.blockTap,
16434       pointGuard: opts?.pointGuard,
16435       sync: opts?.sync,
16436       silent: opts?.silent,
16437       onLongPress: opts?.onLongPress,
16438       longPressMs: opts?.longPressMs,
16439     });
16440     return;
16441   }
16442   target.eventMode = target.eventMode === 'none' ? 'static' : (target.eventMode || 'static');
16443   let armed = false;
16444   let longPressed = false;
16445   let holdTimer: ReturnType<typeof setTimeout> | null = null;
16446   let startX = 0;
16447   let startY = 0;
16448   const clearHold = (): void => {
16449     if (holdTimer !== null) {
16450       clearTimeout(holdTimer);
16451       holdTimer = null;
16452     }
16453   };
16454   target.on('pointerdown', (e: PIXI.FederatedPointerEvent) => {
16455     armed = true;
16456     longPressed = false;
16457     clearHold();
16458     startX = e.global.x;
16459     startY = e.global.y;
16460     if (!opts?.onLongPress) return;
16461     const ms = opts.longPressMs ?? DEFAULT_LONG_PRESS_MS;
16462     holdTimer = setTimeout(() => {
16463       holdTimer = null;
16464       if (!armed) return;
16465       if (opts.guard && !opts.guard()) return;
16466       if (opts.blockTap?.()) return;
16467       longPressed = true;
16468       try {
16469         opts.onLongPress!();
16470       } catch (err) {
16471         console.error('[bindPointerTap longPress]', err);
16472       }

```

```

16473     }, ms);
16474   });
16475   target.on('pointermove', (e: PIXI.FederatedPointerEvent) => {
16476     if (!armed || longPressed || holdTimer == null) return;
16477     const dx = e.global.x - startX;
16478     const dy = e.global.y - startY;
16479     if (dx * dx + dy * dy > HOLD_SLOP * HOLD_SLOP) clearHold();
16480   });
16481   target.on('pointerup', (e: PIXI.FederatedPointerEvent) => {
16482     clearHold();
16483     if (!armed) return;
16484     armed = false;
16485     if (longPressed) return;
16486     const p = e.nativeEvent ? clientEventToDesign(e.nativeEvent) : { x: e.global.x, y: e.global.y };
16487     if (!opts?.silent) playSfx('ui_tap', 40);
16488     run(p.x, p.y);
16489   });
16490   target.on('pointerupoutside', () => {
16491     clearHold();
16492     armed = false;
16493   });
16494   target.on('pointercancel', () => {
16495     clearHold();
16496     armed = false;
16497   });
16498 }
16499 // ===== FILE: src/utils/canvasTapRouter.ts =====
16500 /**
16501  * 微信真机: tap 走 canvas touchstart/touchend + 设计坐标 hitTest (勿混 pointerdown, 会覆盖 _active) 。
16502  * 可选长按: 按住达阈值且位移未超 slop 时触发 onLongPress, 松手不再发 tap。
16503  */
16504 import { Platform } from '@core/PlatformService';
16505 import { playSfx } from '@core/SfxPlayer';
16506 import { clientEventToDesign } from './clientEventToDesign';
16507 import { containsDesignPoint, isOnStage, pickTopmostHit } from './hitTestDesign';
16508 import { deferAfterPointerEvent } from './deferAfterPointer';
16509 import { getTouchCanvas } from './touchCanvas';
16510 const TAP_SLOP = 14;
16511 const DEFAULT_LONG_PRESS_MS = 450;
16512 interface TapBinding {
16513   target: import('pixi.js').Container;
16514   /** 收到本次 tap 的设计坐标。整块热区内部再分区 (编队台选人) 要用。 */
16515   fn: (dx: number, dy: number) => void;
16516   guard?: () => boolean;
16517   blockTap?: () => boolean;
16518   /** 设计坐标额外过滤 (如滚动列表视口外不响应) */
16519   pointGuard?: (dx: number, dy: number) => boolean;
16520   /** 为 true 时在 touchend 内同步执行 (tt.addShortcut 等必须用户手势同步调用的 API) */
16521   sync?: boolean;
16522   silent?: boolean;

```

```

16523   onLongPress?: () => void;
16524   longPressMs?: number;
16525 }
16526 let _installed = false;
16527 let _bindings: TapBinding[] = [];
16528 let _active: { binding: TapBinding; x: number; y: number } | null = null;
16529 let _holdTimer: ReturnType<typeof setTimeout> | null = null;
16530 let _longPressed = false;
16531 let _onStart: EventListener | null = null;
16532 let _onMove: EventListener | null = null;
16533 let _onEnd: EventListener | null = null;
16534 function clearHoldTimer(): void {
16535   if (_holdTimer !== null) {
16536     clearTimeout(_holdTimer);
16537     _holdTimer = null;
16538   }
16539 }
16540 function pickBinding(dx: number, dy: number): TapBinding | null {
16541   // 连 target.parent 都不看够：切场景后旧树整棵脱离舞台，parent 还在，会一直堆着。
16542   _bindings = _bindings.filter((b) => b.target.parent && isOnStage(b.target));
16543   // 禁用态也要参与 hitTest，否则会穿透点到下层按钮（抽卡结果页点「确定」误触底层十连）
16544   const hits = _bindings.filter((b) =>
16545     containsDesignPoint(b.target, dx, dy)
16546     && (!b.pointGuard || b.pointGuard(dx, dy)));
16547   if (!hits.length) return null;
16548   const top = pickTopmostHit(hits.map((b) => b.target), dx, dy);
16549   if (!top) return null;
16550   return hits.find((b) => b.target === top) ?? hits[hits.length - 1];
16551 }
16552 function armLongPress(binding: TapBinding): void {
16553   clearHoldTimer();
16554   if (!binding.onLongPress) return;
16555   const ms = binding.longPressMs ?? DEFAULT_LONG_PRESS_MS;
16556   _holdTimer = setTimeout(() => {
16557     _holdTimer = null;
16558     const act = _active;
16559     if (!act || act.binding !== binding) return;
16560     if (!binding.target.parent) return;
16561     if (binding.guard && !binding.guard()) return;
16562     if (binding.blockTap?.()) return;
16563     _longPressed = true;
16564     try {
16565       binding.onLongPress!();
16566     } catch (err) {
16567       console.error('[canvasTapRouter longPress]', err);
16568     }
16569   }, ms);
16570 }
16571 function ensureInstalled(): void {
16572   if (_installed || !Platform.isMinigame) return;

```

```
16573 const canvas = getTouchCanvas();
16574 if (!canvas?.addEventListener) return;
16575 _onStart = ((e: Event) => {
16576   clearHoldTimer();
16577   _longPressed = false;
16578   const p = clientEventToDesign(e);
16579   const binding = pickBinding(p.x, p.y);
16580   _active = binding ? { binding, x: p.x, y: p.y } : null;
16581   if (binding) armLongPress(binding);
16582 }) as EventListener;
16583 _onMove = ((e: Event) => {
16584   const act = _active;
16585   if (!act || !_longPressed) return;
16586   const p = clientEventToDesign(e);
16587   const dx = p.x - act.x;
16588   const dy = p.y - act.y;
16589   if (dx * dx + dy * dy > TAP_SLOP * TAP_SLOP) {
16590     clearHoldTimer();
16591   }
16592 }) as EventListener;
16593 _onEnd = ((e: Event) => {
16594   clearHoldTimer();
16595   const act = _active;
16596   const wasLong = _longPressed;
16597   _active = null;
16598   _longPressed = false;
16599   if (!act || !wasLong) return;
16600   const p = clientEventToDesign(e);
16601   const movedX = p.x - act.x;
16602   const movedY = p.y - act.y;
16603   if (movedX * movedX + movedY * movedY > TAP_SLOP * TAP_SLOP) return;
16604   // 松手处再取一次最上层。选中后立绘放大，起点热区还钉在旧人身上。
16605   const fresh = pickBinding(p.x, p.y) ?? act.binding;
16606   const b = fresh.target.parent ? fresh : act.binding;
16607   if (!b.target.parent) return;
16608   if (b.guard && !b.guard()) return;
16609   if (b.blockTap?.()) return;
16610   // 必须在 touchend 里同步播。丢进 setTimeout 微信当非手势，直接静音。
16611   if (!b.silent) playSfx('ui_tap', 40);
16612   if (b.sync) {
16613     try { b.fn(p.x, p.y); } catch (err) { console.error('[canvasTapRouter sync]', err); }
16614   } else {
16615     deferAfterPointerEvent(() => b.fn(p.x, p.y));
16616   }
16617 }) as EventListener;
16618 const onCancel = (): void => {
16619   clearHoldTimer();
16620   _active = null;
16621   _longPressed = false;
16622 };
```

```

16623 canvas.addEventListener('touchstart', _onStart, { passive: true });
16624 canvas.addEventListener('touchmove', _onMove, { passive: true });
16625 canvas.addEventListener('touchend', _onEnd);
16626 canvas.addEventListener('touchcancel', onCancel);
16627 _installed = true;
16628 }
16629 export function registerCanvasTap(binding: TapBinding): () => void {
16630   ensureInstalled();
16631   _bindings.push(binding);
16632   return () => {
16633     const i = _bindings.indexOf(binding);
16634     if (i !== -1) _bindings.splice(i, 1);
16635   };
16636 }
16637 // ===== FILE: src/utils/clientEventToDesign.ts =====
16638 /**
16639  * 微信 touch / pointer → 设计坐标 (750 宽) 。
16640  * 统一规则见 @/minigame/index.ts; 对齐 game2D_huahua BoardView._rawToLocal。
16641  */
16642 import * as PIXI from 'pixi.js';
16643 import { Game } from '@core/Game';
16644 import { designPointToContainerLocal } from './hitTestDesign';
16645 export function clientEventToDesign(e: unknown): { x: number; y: number } {
16646   const ev = e as {
16647     clientX?: number;
16648     clientY?: number;
16649     x?: number;
16650     y?: number;
16651     touches?: Array<{ clientX?: number; clientY?: number }>;
16652     changedTouches?: Array<{ clientX?: number; clientY?: number }>;
16653   };
16654   const t0 = ev.changedTouches?.[0] ?? ev.touches?.[0];
16655   const cx = ev.clientX ?? t0?.clientX ?? ev.x ?? 0;
16656   const cy = ev.clientY ?? t0?.clientY ?? ev.y ?? 0;
16657   // 对齐 game2D_huahua: clientX(逻辑像素) → 设计坐标, 单一比例 designWidth/screenWidth。
16658   // 不经 EventSystem patch / toLocal, 避免依赖 worldTransform 是否最新。
16659   const ratio = Game.designWidth / Game.screenWidth;
16660   return { x: cx * ratio, y: cy * ratio };
16661 }
16662 export function designPointToLocal(target: PIXI.Container, dx: number, dy: number): PIXI.Point {
16663   return designPointToContainerLocal(target, dx, dy);
16664 }
16665 /** touch/pointer 事件 → 某容器本地设计坐标 (棋盘拖拽 / hitTest 共用) */
16666 export function designEventToLocal(target: PIXI.Container, e: unknown): { x: number; y: number } {
16667   const design = clientEventToDesign(e);
16668   const local = designPointToLocal(target, design.x, design.y);
16669   return { x: local.x, y: local.y };
16670 }
16671 // ===== FILE: src/utils/deferAfterPointer.ts =====
16672 /**

```

```
16673 * 推迟到当前指针事件派发完成后再执行。
16674 * 避免 pointertap 内切场景/destroy 节点后, Pixi pointerup 仍遍历 pressTargets 报 null.scale。
16675 */
16676 import { Game } from '@core/Game';
16677 import { Platform } from '@core/PlatformService';
16678 let _rafId = 0;
16679 const _queue: Array<() => void> = [];
16680 export function deferAfterPointerEvent(fn: () => void): void {
16681   _queue.push(fn);
16682   if (_rafId !== 0) return;
16683   const flush = () => {
16684     _rafId = 0;
16685     const batch = _queue.splice(0);
16686     for (const run of batch) {
16687       try {
16688         run();
16689       } catch (err) {
16690         console.error('[deferAfterPointer]', err);
16691       }
16692     }
16693   };
16694   _rafId = -1;
16695   // 开发者工具也走 setTimeout: 微信模拟器里 ticker.addOnce 会偶发丢,
16696   // 点下去像没反应。真机同样不能靠 addOnce (见 deferNextFrame 注释)。
16697   if (Platform.isMinigame) {
16698     setTimeout(flush, 0);
16699     return;
16700   }
16701   if (Game.ticker?.started) {
16702     Game.ticker.addOnce(flush);
16703     return;
16704   }
16705   _rafId = requestAnimationFrame(flush);
16706 }
16707 /** 推迟到下一帧 (场景 switchTo 后的 build 专用, 避免与 Pixi 挂载同帧冲突) 。 */
16708 export function deferNextFrame(fn: () => void): void {
16709   const run = () => {
16710     try {
16711       fn();
16712     } catch (err) {
16713       console.error('[deferNextFrame]', err);
16714     }
16715   };
16716   // 真机 pointerup→setTimeout(switchTo) 后 ticker.addOnce 常不触发, build 永远不执行 → 黑屏
16717   if (Platform.isMinigame && !Platform.isDevtools) {
16718     setTimeout(run, 0);
16719     return;
16720   }
16721   if (Game.ticker?.started) {
16722     Game.ticker.addOnce(run);
```

```
16723     return;
16724 }
16725 requestAnimationFrame(run);
16726 }
16727 // ===== FILE: src/utils/hitTestDesign.ts =====
16728 /**
16729  * 设计坐标 hitTest (750 布局) , 供 canvasTapRouter / ScrollList / 编队列表共用。
16730  *
16731  * 注意: 小游戏 tap 不走 PIXI 原生 hitTest, 必须在此自行尊重祖先 mask。
16732  * 否则滚动列表视觉上被裁切后, 仍会抢走顶栏「返回」等按钮的点击。
16733  */
16734 import * as PIXI from 'pixi.js';
16735 import { Game } from '@core/Game';
16736 /** 设计坐标 → 容器本地 (含父级 scale/pivot, 地图 cover 缩放后点击才准) */
16737 export function designPointToContainerLocal(target: PIXI.Container, dx: number, dy: number): PIXI...
16738     return target.toLocal(new PIXI.Point(dx, dy), Game.stage);
16739 }
16740 function localContains(
16741     local: PIXI.Point,
16742     ha: PIXI.IHitArea | null | undefined,
16743     fallback: PIXI.Container,
16744 ): boolean {
16745     if (ha instanceof PIXI.Rectangle) {
16746         return local.x >= ha.x && local.x <= ha.x + ha.width
16747             && local.y >= ha.y && local.y <= ha.y + ha.height;
16748     }
16749     if (ha instanceof PIXI.Circle) {
16750         const dxl = local.x - ha.x;
16751         const dyl = local.y - ha.y;
16752         return dxl * dxl + dyl * dyl <= ha.radius * ha.radius;
16753     }
16754     if (ha instanceof PIXI.RoundedRectangle) {
16755         return ha.contains(local.x, local.y);
16756     }
16757     if (ha && typeof (ha as { contains?: unknown }).contains === 'function') {
16758         return (ha as { contains: (x: number, y: number) => boolean }).contains(local.x, local.y);
16759     }
16760     const b = fallback.getLocalBounds();
16761     return local.x >= b.x && local.x <= b.x + b.width
16762         && local.y >= b.y && local.y <= b.y + b.height;
16763 }
16764 /**
16765  * 祖先链上若有 mask, 设计点必须落在 mask 形状内 (对齐 PIXI 视觉裁切) 。
16766  * 滚动列表用 Graphics.drawRect 做视口 mask 时, 滚出视口的卡片不再可点。
16767  */
16768 /** mask 可能是容器本身, 也可能是包了一层的 MaskData */
16769 function resolveMaskObject(mask: PIXI.Container | PIXI.MaskData | null): PIXI.Container | null {
16770     if (!mask) return null;
16771     const obj = mask instanceof PIXI.MaskData ? mask.maskObject : mask;
16772     return (obj as PIXI.Container | null) ?? null;
```

```

16773 }
16774 function insideAncestorMasks(target: PIXI.Container, dx: number, dy: number): boolean {
16775     let cur: PIXI.Container | null = target;
16776     while (cur) {
16777         const maskObj = resolveMaskObject(cur.mask);
16778         if (maskObj) {
16779             const local = designPointToContainerLocal(maskObj, dx, dy);
16780             if (!localContains(local, maskObj.hitArea, maskObj)) return false;
16781         }
16782         cur = cur.parent;
16783     }
16784     return true;
16785 }
16786 /**
16787  * 节点是否还挂在当前舞台上。
16788  *
16789  * 切场景只做 stage.removeChild(旧场景.container)，旧场景内部的节点 parent 仍在、
16790  * visible 仍是 true，Pixa 的 worldVisible 也照样返回 true——它只看 visible 标志，
16791  * 不管这棵树有没有连到舞台。于是上一局战斗的选卡热区会继续在村口抢点击，
16792  * 把村口点人的整块热区压住（点人无反应、只有没被盖住的那个能选）。
16793  */
16794 export function isOnStage(target: PIXI.Container): boolean {
16795     const stage = Game.stage;
16796     if (!stage) return true;
16797     let cur: PIXI.Container = target;
16798     while (cur.parent) cur = cur.parent as PIXI.Container;
16799     return cur === stage;
16800 }
16801 export function containsDesignPoint(target: PIXI.Container, dx: number, dy: number): boolean {
16802     if (!target.parent || !target.visible || target.worldVisible === false) return false;
16803     if (target.eventMode === 'none') return false;
16804     if (!isOnStage(target)) return false;
16805     if (!insideAncestorMasks(target, dx, dy)) return false;
16806     const local = designPointToContainerLocal(target, dx, dy);
16807     return localContains(local, target.hitArea, target);
16808 }
16809 /** 从候选容器里取最上层命中项（对齐 Pixi 绘制顺序：后绘制的在上） */
16810 export function pickTopmostHit(
16811     candidates: PIXI.Container[],
16812     dx: number,
16813     dy: number,
16814 ): PIXI.Container | null {
16815     let best: PIXI.Container | null = null;
16816     let bestPath: number[] | null = null;
16817     for (const target of candidates) {
16818         if (!containsDesignPoint(target, dx, dy)) continue;
16819         const path = worldPath(target);
16820         if (!bestPath || comparePath(path, bestPath) >= 0) {
16821             best = target;
16822             bestPath = path;

```



```

16823     }
16824   }
16825   return best;
16826 }
16827 /**
16828  * 从根到叶的子节点索引链。
16829  *
16830  * 必须保留成数组逐层比，不能压成一个数。曾经压成 `order * 1000 + index` 从叶往根
16831  * 累加，等于把叶节点的索引放到了最高位：Overlay 层的 GM 按钮算出 3002，战斗场景里
16832  * 埋得更深的点怪热区算出 15001，热区反而赢了——点 GM 按钮弹出的是怪物说明。
16833  * 只要下层子树够深，它的叶节点就能压过顶层容器，「Overlay 永远在最上层」这个前提
16834  * 就被算法本身破坏了。
16835  */
16836 function worldPath(target: PIXI.Container): number[] {
16837   const path: number[] = [];
16838   let cur: PIXI.Container | null = target;
16839   while (cur?.parent) {
16840     path.push(cur.parent.getChildIndex(cur));
16841     cur = cur.parent;
16842   }
16843   return path.reverse();
16844 }
16845 /** 顶层先分胜负，同层再往下比；祖先相同时更深的那个在上 */
16846 function comparePath(a: readonly number[], b: readonly number[]): number {
16847   const n = Math.max(a.length, b.length);
16848   for (let i = 0; i < n; i++) {
16849     const av = a[i] ?? -1;
16850     const bv = b[i] ?? -1;
16851     if (av !== bv) return av - bv;
16852   }
16853   return 0;
16854 }
16855 // ===== FILE: src/utils/touchCanvas.ts =====
16856 /**
16857  * 微信双 canvas: TouchEvent 桥接挂在 GameGlobal.canvas (pixi-primary) 。
16858  */
16859 import { Game } from '@core/Game';
16860 declare const GameGlobal: { canvas?: TouchCanvasLike } | undefined;
16861 export type TouchCanvasLike = {
16862   addEventListener: (type: string, handler: EventListener, options?: boolean | AddEventListenerOp...
16863   removeEventListener: (type: string, handler: EventListener, options?: boolean | AddEventListene...
16864 };
16865 export function getTouchCanvas(): TouchCanvasLike {
16866   const bridged = GameGlobal?.canvas;
16867   if (bridged && typeof bridged.addEventListener === 'function') {
16868     return bridged;
16869   }
16870   return Game.app.view as unknown as TouchCanvasLike;
16871 }
16872 // ===== FILE: cloudfunctions/cunkou-api/index.js =====

```

```
16873 const { handleLogin } = require('./lib/auth');
16874 const { handlePull, handlePush } = require('./lib/save');
16875 const { respond, parseEvent, preflight } = require('./lib/http');
16876 const { getGameKey, getScopedGameKey } = require('./lib/config');
16877 const ROUTES = {
16878   'GET /health': async () => ({
16879     ok: true,
16880     gameKey: getGameKey(),
16881     scopedGameKeys: { wx: getScopedGameKey('wx'), dy: getScopedGameKey('dy'), tap: getScopedGameK...
16882     ts: Date.now(),
16883   }),
16884   'POST /health': async () => ({
16885     ok: true,
16886     gameKey: getGameKey(),
16887     scopedGameKeys: { wx: getScopedGameKey('wx'), dy: getScopedGameKey('dy'), tap: getScopedGameK...
16888     ts: Date.now(),
16889   }),
16890   'POST /login': handleLogin,
16891   'POST /save/pull': handlePull,
16892   'POST /save/push': handlePush,
16893 };
16894 exports.main = async (event, context) => {
16895   try {
16896     if (event && event.httpMethod === 'OPTIONS') {
16897       return preflight();
16898     }
16899     const req = parseEvent(event);
16900     const key = `${req.method} ${req.path}`;
16901     const handler = ROUTES[key];
16902     if (!handler) {
16903       return respond(404, { ok: false, code: 'NOT_FOUND', error: `no route: ${key}` });
16904     }
16905     const result = await handler(req, context);
16906     if (result && typeof result === 'object' && 'statusCode' in result) {
16907       return result;
16908     }
16909     return respond(200, { ok: true, data: result });
16910   } catch (error) {
16911     const code = error && error.code ? error.code : 'INTERNAL';
16912     const status = error && error.status ? error.status : 500;
16913     const message = (error && error.message) || String(error);
16914     console.error('[cunkou-api] error:', code, message, error && error.stack);
16915     const out = { ok: false, code, error: message };
16916     if (error && error.data !== undefined) {
16917       out.data = error.data;
16918     }
16919     return respond(status, out);
16920   }
16921 };
16922 // ===== FILE: cloudfunctions/cunkou-api/lib/auth.js =====
```

```
16923 const jwt = require('jsonwebtoken');
16924 const { httpError } = require('./http');
16925 const {
16926   getGameKey,
16927   getScopedGameKey,
16928   gameKeyUpper,
16929   getJwtSecret: readJwtSecret,
16930   getTtlSec,
16931   getPlatformCredential,
16932 } = require('./config');
16933 const SUPPORTED_PLATFORMS = new Set(['wx', 'dy', 'tap', 'anon']);
16934 function getJwtSecret() {
16935   const secret = readJwtSecret();
16936   if (!secret) {
16937     throw httpError(500, 'NO_JWT_SECRET', `${gameKeyUpper()}_JWT_SECRET 未配置`);
16938   }
16939   return secret;
16940 }
16941 async function handleLogin(req) {
16942   const body = req.body || {};
16943   const platform = String(body.platform || '').toLowerCase();
16944   if (!SUPPORTED_PLATFORMS.has(platform)) {
16945     throw httpError(400, 'BAD_PLATFORM', `unsupported platform: ${platform}`);
16946   }
16947   let platformUid = '';
16948   if (platform === 'wx') {
16949     const wxSession = await wxCode2Session(body.code);
16950     platformUid = wxSession.openid;
16951   } else if (platform === 'dy') {
16952     platformUid = await ttCode2Openid(body.code);
16953   } else if (platform === 'tap') {
16954     const tapSession = await tapCode2Session(body.code);
16955     platformUid = tapSession.openid;
16956   } else if (platform === 'anon') {
16957     const id = String(body.anonId || '').trim();
16958     if (!id) throw httpError(400, 'NO_ANON_ID', 'anonId 缺失');
16959     if (!/^[A-Za-z0-9_-\.:]{8,128}$/.test(id)) {
16960       throw httpError(400, 'BAD_ANON_ID', 'anonId 非法');
16961     }
16962     platformUid = id;
16963   }
16964   const userId = `${platform}:${platformUid}`;
16965   const ttlSec = getTtlSec();
16966   const scopedGameKey = getScopedGameKey(platform);
16967   const now = Math.floor(Date.now() / 1000);
16968   const token = jwt.sign(
16969     { sub: userId, plt: platform, gk: scopedGameKey, iat: now },
16970     getJwtSecret(),
16971     { expiresIn: ttlSec },
16972   );
```

```
16973   return {
16974     token,
16975     userId,
16976     platform,
16977     gameKey: scopedGameKey,
16978     baseGameKey: getGameKey(),
16979     expiresAt: (now + ttlSec) * 1000,
16980     ttlSec,
16981   };
16982 }
16983 function requireUser(req) {
16984   const authHeader = (req.headers && req.headers.authorization) || '';
16985   const match = /^Bearer\s+(.+)$/i.exec(authHeader);
16986   if (!match) throw httpError(401, 'NO_TOKEN', '缺少 Authorization: Bearer <token>');
16987   let payload;
16988   try {
16989     payload = jwt.verify(match[1].trim(), getJwtSecret());
16990   } catch (error) {
16991     throw httpError(401, 'BAD_TOKEN', error && error.message ? error.message : 'token 无效');
16992   }
16993   const userId = payload && payload.sub;
16994   if (!userId || typeof userId !== 'string' || !userId.includes(':')) {
16995     throw httpError(401, 'BAD_TOKEN', 'token sub 非法');
16996   }
16997   const platform = payload.plt || userId.split(':')[0];
16998   const currentGameKey = getScopedGameKey(platform);
16999   if (payload.gk && payload.gk !== currentGameKey) {
17000     throw httpError(
17001       401,
17002       'BAD_TOKEN',
17003       `token gameKey=${payload.gk} 与当前 scoped GAME_KEY=${currentGameKey} 不匹配`,
17004     );
17005   }
17006   return { userId, platform, gameKey: currentGameKey };
17007 }
17008 async function wxCode2Session(code) {
17009   const appid = getPlatformCredential('wx', 'APPID');
17010   const secret = getPlatformCredential('wx', 'SECRET');
17011   if (!appid || !secret) throw httpError(500, 'NO_WX_CFG', `${gameKeyUpper()}_WX_APPID/${gameKeyU...`);
17012   if (!code) throw httpError(400, 'NO_CODE', 'wx code 缺失');
17013   const url = `https://api.weixin.qq.com/sns/jscode2session?appid=${encodeURIComponent(appid)}&se...`;
17014   const data = await httpGetJson(url);
17015   if (!data || !data.openid) {
17016     throw httpError(401, 'WX_LOGIN_FAIL', `wx code2session 失败: ${JSON.stringify(data || {})}`);
17017   }
17018   return {
17019     openid: data.openid,
17020     session_key: typeof data.session_key === 'string' ? data.session_key : '',
17021   };
17022 }
```

```
17023 async function ttCode2Openid(code) {
17024   const appid = getPlatformCredential('tt', 'APPID');
17025   const secret = getPlatformCredential('tt', 'SECRET');
17026   if (!appid || !secret) throw httpError(500, 'NO_TT_CFG', `${gameKeyUpper()}_TT_APPID/${gameKeyU...
17027   if (!code) throw httpError(400, 'NO_CODE', 'dy code 缺失');
17028   const data = await httpPostJson('https://developer.toutiao.com/api/apps/v2/jscode2session', { a...
17029   if (!data || data.err_no !== 0 || !data.data || !data.data.openid) {
17030     throw httpError(401, 'TT_LOGIN_FAIL', `dy code2session 失败: ${JSON.stringify(data || {})}`);
17031   }
17032   return data.data.openid;
17033 }
17034 async function tapCode2Session(code) {
17035   const appid = getPlatformCredential('tap', 'APPID');
17036   const secret = getPlatformCredential('tap', 'SECRET');
17037   if (!appid || !secret) throw httpError(500, 'NO_TAP_CFG', `${gameKeyUpper()}_TAP_APPID/${gameKe...
17038   if (!code) throw httpError(400, 'NO_CODE', 'tap code 缺失');
17039   const url = `https://cloud-miniapp.tapapis.cn/auth/v1/jscode2session?appid=${encodeURIComponent...
17040   const data = await httpGetJson(url);
17041   if (!data || !data.openid || data.errcode) {
17042     throw httpError(401, 'TAP_LOGIN_FAIL', `tap code2session 失败: ${JSON.stringify(data || {})}`);
17043   }
17044   return {
17045     openid: data.openid,
17046     session_key: typeof data.session_key === 'string' ? data.session_key : '',
17047   };
17048 }
17049 function httpGetJson(url) {
17050   return httpRequestJson(url, 'GET');
17051 }
17052 function httpPostJson(url, body) {
17053   return httpRequestJson(url, 'POST', body);
17054 }
17055 function httpRequestJson(url, method, body) {
17056   if (typeof fetch !== 'function') {
17057     return Promise.reject(httpError(500, 'NO_FETCH', '当前 Node 运行时不支持 fetch, 请使用 Node 18+'));
17058   }
17059   const opts = { method, headers: { 'Content-Type': 'application/json' } };
17060   if (body !== undefined) {
17061     opts.body = JSON.stringify(body);
17062   }
17063   return fetch(url, opts).then(async (res) => {
17064     const text = await res.text();
17065     try {
17066       return JSON.parse(text);
17067     } catch (_) {
17068       return { _raw: text };
17069     }
17070   });
17071 }
17072 module.exports = {
```

```
17073   handleLogin,
17074   requireUser,
17075 };
17076 // ===== FILE: cloudfunctions/cunkou-api/lib/config.js =====
17077 const DEFAULT_GAME_KEY = 'cunkou';
17078 const DEFAULT_TTL_SEC = 7 * 24 * 3600;
17079 const DEFAULT_MAX_BYTES = 256 * 1024;
17080 /** 后端 platform 字段 → 命名空间段（与客户端 gameKeyScope 一致） */
17081 function getPlatformScope(platform) {
17082   const p = String(platform || '').toLowerCase();
17083   if (p === 'dy' || p === 'tt' || p === 'douyin') return 'tt';
17084   if (p === 'tap' || p === 'taptap') return 'tap';
17085   return null;
17086 }
17087 function getGameKey() {
17088   const v = String(process.env.GAME_KEY || '').trim().toLowerCase();
17089   if (!v) return DEFAULT_GAME_KEY;
17090   if (!/^[a-z][a-z0-9_]{0,31}$/.test(v)) {
17091     throw new Error(`非法 GAME_KEY: ${v}`);
17092   }
17093   return v;
17094 }
17095 /** 存档 / 集合 / JWT gk 使用的命名空间: 微信 cunkou, 抖音 cunkou_tt */
17096 function getScopedGameKey(platform) {
17097   const base = getGameKey();
17098   const scope = getPlatformScope(platform);
17099   return scope ? `${base}_${scope}` : base;
17100 }
17101 function scopedKeyUpper(platform) {
17102   return getScopedGameKey(platform).toUpperCase().replace(/^[A-Z0-9]/g, '_');
17103 }
17104 function gameKeyUpper() {
17105   return getGameKey().toUpperCase().replace(/^[A-Z0-9]/g, '_');
17106 }
17107 function readEnvPrefer(...keys) {
17108   for (const key of keys) {
17109     const value = process.env[key];
17110     if (value !== undefined && value !== null && String(value).length > 0) {
17111       return String(value);
17112     }
17113   }
17114   return '';
17115 }
17116 function getCollectionName(suffix, platform) {
17117   const normalizedSuffix = String(suffix || '').replace(/^_+/, '');
17118   const overrideKey = `${scopedKeyUpper(platform)}_${normalizedSuffix.toUpperCase()}_COLLECTION`;
17119   const override = process.env[overrideKey];
17120   if (override) {
17121     return String(override);
17122   }
```

```
17123   return `${getScopedGameKey(platform)}_${normalizedSuffix}`;
17124 }
17125 function getJwtSecret() {
17126   return readEnvPrefer(`${gameKeyUpper()}_JWT_SECRET`);
17127 }
17128 function getTtlSec() {
17129   const raw = readEnvPrefer(`${gameKeyUpper()}_TOKEN_TTL_SEC`);
17130   const v = Number(raw);
17131   return Number.isFinite(v) && v > 0 ? Math.floor(v) : DEFAULT_TTL_SEC;
17132 }
17133 function getMaxBytes() {
17134   const raw = readEnvPrefer(`${gameKeyUpper()}_SAVE_MAX_BYTES`);
17135   const v = Number(raw);
17136   return Number.isFinite(v) && v > 0 ? Math.floor(v) : DEFAULT_MAX_BYTES;
17137 }
17138 function getPlatformCredential(platform, field) {
17139   const upper = gameKeyUpper();
17140   const platformUpper = platform.toUpperCase();
17141   return readEnvPrefer(`${upper}_${platformUpper}_${field}`);
17142 }
17143 module.exports = {
17144   getGameKey,
17145   getPlatformScope,
17146   getScopedGameKey,
17147   scopedKeyUpper,
17148   gameKeyUpper,
17149   getCollectionName,
17150   getJwtSecret,
17151   getTtlSec,
17152   getMaxBytes,
17153   getPlatformCredential,
17154 };
17155 // ===== FILE: cloudfunctions/cunkou-api/lib/db.js =====
17156 const tcb = require('@cloudbase/node-sdk');
17157 const { getCollectionName } = require('./config');
17158 let app = null;
17159 function getApp() {
17160   if (app) return app;
17161   app = tcb.init({
17162     env: process.env.TCB_ENV || tcb.SYMBOL_CURRENT_ENV,
17163   });
17164   return app;
17165 }
17166 function getDb() {
17167   return getApp().database();
17168 }
17169 function getCollection(platform) {
17170   return getDb().collection(getCollectionName('playerData', platform));
17171 }
17172 module.exports = {
```

```
17173   getApp,
17174   getDb,
17175   getCollection,
17176 };
17177 // ===== FILE: cloudfunctions/cunkou-api/lib/http.js =====
17178 const { getGameKey } = require('./config');
17179 const CORS_HEADERS = {
17180   'Access-Control-Allow-Origin': '*',
17181   'Access-Control-Allow-Methods': 'GET,POST,OPTIONS',
17182   'Access-Control-Allow-Headers': 'Content-Type,Authorization',
17183   'Access-Control-Max-Age': '86400',
17184 };
17185 function respond(statusCode, body, extraHeaders = {}) {
17186   return {
17187     statusCode,
17188     headers: {
17189       'Content-Type': 'application/json; charset=utf-8',
17190       ...CORS_HEADERS,
17191       ...extraHeaders,
17192     },
17193     body: typeof body === 'string' ? body : JSON.stringify(body),
17194     isBase64Encoded: false,
17195   };
17196 }
17197 function preflight() {
17198   return {
17199     statusCode: 204,
17200     headers: { ...CORS_HEADERS },
17201     body: "",
17202     isBase64Encoded: false,
17203   };
17204 }
17205 function parseEvent(event) {
17206   event = event || {};
17207   if (event.httpMethod) {
17208     const method = String(event.httpMethod).toUpperCase();
17209     let rawBody = event.body || "";
17210     if (event.isBase64Encoded && rawBody) {
17211       try {
17212         rawBody = Buffer.from(rawBody, 'base64').toString('utf8');
17213       } catch (_) {}
17214     }
17215     let body = {};
17216     if (rawBody) {
17217       try {
17218         body = JSON.parse(rawBody);
17219       } catch (_) {}
17220       body = {};
17221     }
17222   }
```



```
17223   return {
17224     method,
17225     path: normalizePath(event.path || '/'),
17226     body,
17227     headers: lowercaseHeaders(event.headers || {}),
17228     query: event.queryStringParameters || {},
17229     raw: event,
17230   };
17231 }
17232 const action = String(event.action || "").replace(/^\//+/, "");
17233 return {
17234   method: 'POST',
17235   path: action ? `/${action}` : '/',
17236   body: event.body || {},
17237   headers: lowercaseHeaders(event.headers || {}),
17238   query: {},
17239   raw: event,
17240 };
17241 }
17242 function normalizePath(path) {
17243   if (!path) return '/';
17244   let p = String(path);
17245   if (!p.startsWith('/')) p = `/${p}`;
17246   const apiPrefix = `${getGameKey()}-api`;
17247   // HTTP 挂载路径可能是 cunkou-api (驼峰) , GAME_KEY 归一化后是小写 pettower
17248   p = p.replace(new RegExp(`^(?:${escapeRegExp(apiPrefix)})(?=/|$\s|i)`), "");
17249   if (p === "") p = '/';
17250   if (p.length > 1 && p.endsWith('/')) p = p.slice(0, -1);
17251   return p;
17252 }
17253 function lowercaseHeaders(headers) {
17254   const out = {};
17255   for (const key of Object.keys(headers || {})) {
17256     out[key.toLowerCase()] = headers[key];
17257   }
17258   return out;
17259 }
17260 function httpError(status, code, message) {
17261   const err = new Error(message || code);
17262   err.status = status;
17263   err.code = code;
17264   return err;
17265 }
17266 function escapeRegExp(text) {
17267   return String(text).replace(/[\.*+?^${}()|[\]\]/g, '\\$&');
17268 }
17269 module.exports = {
17270   respond,
17271   preflight,
17272   parseEvent,
```

```
17273   httpError,
17274 };
17275 // ===== FILE: cloudfunctions/cunkou-api/lib/save.js =====
17276 const { httpError } = require('./http');
17277 const { requireUser } = require('./auth');
17278 const { getCollection } = require('./db');
17279 const { getMaxBytes } = require('./config');
17280 function isCollectionNotExistError(error) {
17281   const msg = String((error && error.message) || error || "");
17282   return /collection not exist|DATABASE_COLLECTION_NOT_EXIST|ResourceNotFound|Db or Table not exi...
17283 }
17284 function emptyPullResult(userId, platform) {
17285   return {
17286     userId,
17287     platform,
17288     exists: false,
17289     schemaVersion: 0,
17290     updatedAt: 0,
17291     payload: {},
17292     payloadKeys: [],
17293   };
17294 }
17295 async function handlePull(req) {
17296   const { userId, platform } = requireUser(req);
17297   const col = getCollection(platform);
17298   let res;
17299   try {
17300     res = await col.where({ userId }).limit(1).get();
17301   } catch (error) {
17302     if (isCollectionNotExistError(error)) {
17303       return emptyPullResult(userId, platform);
17304     }
17305     throw error;
17306   }
17307   const doc = (res && Array.isArray(res.data) && res.data[0]) || null;
17308   if (!doc) {
17309     return {
17310       userId,
17311       platform,
17312       exists: false,
17313       schemaVersion: 0,
17314       updatedAt: 0,
17315       payload: {},
17316       payloadKeys: [],
17317     };
17318   }
17319   return {
17320     userId,
17321     platform,
17322     exists: true,
```

```
17323   schemaVersion: doc.schemaVersion || 0,
17324   updatedAt: doc.updatedAt || 0,
17325   payload: doc.payload || {},
17326   payloadKeys: Array.isArray(doc.payloadKeys) ? doc.payloadKeys : Object.keys(doc.payload || {}),
17327   clientFingerprint: doc.clientFingerprint || "",
17328 };
17329 }
17330 async function handlePush(req) {
17331   const { userId, platform } = requireUser(req);
17332   const body = req.body || {};
17333   const schemaVersion = Number(body.schemaVersion);
17334   const updatedAt = Number(body.updatedAt);
17335   const baseRemoteUpdatedAt = Number(body.baseRemoteUpdatedAt || 0);
17336   const payload = body.payload;
17337   const clientFingerprint = String(body.clientFingerprint || "").slice(0, 200);
17338   const force = body.force === true;
17339   if (!Number.isFinite(schemaVersion) || schemaVersion <= 0) {
17340     throw httpError(400, 'BAD_SCHEMA', 'schemaVersion 非法');
17341   }
17342   if (!Number.isFinite(updatedAt) || updatedAt <= 0) {
17343     throw httpError(400, 'BAD_UPDATED_AT', 'updatedAt 非法');
17344   }
17345   if (!Number.isFinite(baseRemoteUpdatedAt) || baseRemoteUpdatedAt < 0) {
17346     throw httpError(400, 'BAD_BASE_REMOTE_UPDATED_AT', 'baseRemoteUpdatedAt 非法');
17347   }
17348   if (!payload || typeof payload !== 'object' || Array.isArray(payload)) {
17349     throw httpError(400, 'BAD_PAYLOAD', 'payload 必须是 object');
17350   }
17351   const size = Buffer.byteLength(JSON.stringify(payload), 'utf8');
17352   const maxBytes = getMaxBytes();
17353   if (size > maxBytes) {
17354     throw httpError(413, 'PAYLOAD_TOO_LARGE', `payload 超限: ${size}B > ${maxBytes}B`);
17355   }
17356   const payloadKeys = [];
17357   for (const key of Object.keys(payload)) {
17358     if (typeof payload[key] !== 'string') {
17359       throw httpError(400, 'BAD_PAYLOAD_VALUE', `payload[${key}] 必须是字符串`);
17360     }
17361     payloadKeys.push(key);
17362   }
17363   const col = getCollection(platform);
17364   let existing = null;
17365   try {
17366     const existingRes = await col.where({ userId }).limit(1).get();
17367     existing = (existingRes && Array.isArray(existingRes.data) && existingRes.data[0]) || null;
17368   } catch (error) {
17369     if (isCollectionNotExistError(error)) {
17370       throw error;
17371     }
17372   }
```

```
17373 if (existing && !force) {
17374   const prevUpdatedAt = Number(existing.updatedAt) || 0;
17375   if (updatedAt < prevUpdatedAt || baseRemoteUpdatedAt < prevUpdatedAt) {
17376     throw Object.assign(
17377       httpError(409, 'STALE_UPDATE', `服务端已有更新版本 remote=${prevUpdatedAt} > local=${updatedAt}, b...
17378     {
17379       data: {
17380         remote: {
17381           schemaVersion: existing.schemaVersion || 0,
17382           updatedAt: prevUpdatedAt,
17383           payload: existing.payload || {},
17384           payloadKeys: Array.isArray(existing.payloadKeys)
17385             ? existing.payloadKeys
17386             : Object.keys(existing.payload || {}),
17387         },
17388       },
17389     },
17390   );
17391   }
17392 }
17393 const now = Date.now();
17394 const docData = {
17395   userId,
17396   platform,
17397   schemaVersion,
17398   updatedAt,
17399   baseRemoteUpdatedAt,
17400   clientFingerprint,
17401   payload,
17402   payloadKeys,
17403   lastWriteAt: now,
17404 };
17405 if (existing && existing._id) {
17406   await col.doc(existing._id).update(docData);
17407   return { userId, updatedAt, savedAt: now, mode: 'update', sizeBytes: size };
17408 }
17409 const addRes = await col.add(docData);
17410 return {
17411   userId,
17412   updatedAt,
17413   savedAt: now,
17414   mode: 'insert',
17415   sizeBytes: size,
17416   _id: addRes && (addRes.id || addRes._id),
17417 };
17418 }
17419 module.exports = {
17420   handlePull,
17421   handlePush,
17422 };
```